



ΧΑΡΟΚΟΠΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ
HAROKOPIO UNIVERSITY

Εξόρυξη Δεδομένων και Συστήματα Συστάσεων

Τσιλιμπώκος Σπυρίδων (ais25118)

ΑΘΗΝΑ 2025-2026



Περιεχόμενα

Ο Στόχος της Εργασίας (The Vision)	5
Αρχείο KNINE workflow	5
Επεξήγηση Flow	5
1. Ενοποίηση Δεδομένων (Data Integration)	5
2. Καθαρισμός Δεδομένων (Data Cleaning).....	5
3. Μηχανική Χαρακτηριστικών (Feature Engineering)	6
4. Προετοιμασία για Εκπαίδευση (Partitioning)	6
Outliers.....	7
Decision Tree	8
1. Αυτοματοποιημένος Πειραματισμός (Loop Start)	8
2. Παράλληλη Αξιολόγηση	8
3. Συλλογή και Ενοποίηση Αποτελεσμάτων (Aggregation)	9
4. Επιλογή Βέλτιστου Μοντέλου (Selection).....	9
Gradient Boosted Trees	9
1. Αναβάθμιση του Μηχανισμού Αναζήτησης (Optimization Loop).....	9
2. Η Λογική του Αλγορίθμου (Learner)	9
3. Έλεγχος Υπερ-εκπαίδευσης (The Sandwich Evaluation)	10
4. Τελική Επιλογή (Ranking)	10
Naive Bayes.....	10
1. Εφαρμογή Ενιαίας Μεθοδολογίας	10
2. Τι ρυθμίσουμε; (The Tuning Parameter)	10
Random Forest	11
1. Εφαρμογή Ενιαίας Μεθοδολογίας	11
2. Τι ρυθμίσουμε; (The Tuning Parameter)	11
Normalization	12
Για τα One to Many warnings	12
1. Κωδικοποίηση Κατηγορικών Μεταβλητών (One-Hot Encoding)	13
2. Κανονικοποίηση (Normalization): Το Κρισιμότερο Βήμα.....	13
3. Τι Αποφύγαμε και Γιατί (Optimization).....	13
SVM.....	14
1. Στοχευμένη Βελτιστοποίηση (Optimization Loop).....	14
2. Η Γεωμετρική Προσέγγιση (The Learner)	14
3. Έλεγχος Υπερ-προσαρμογής (Double Validation)	14
4. Διαχείριση Πόρων (Resource Management)	15



RProp MLP	15
1. Ο Χρόνος ως Παράμετρος (Epochs Optimization)	15
2. Η Αρχιτεκτονική του Δικτύου (The Learner)	15
3. Η Αποκάλυψη του Overfitting (Validation Strategy)	15
4. Τελική Κρίση (Conclusion)	16
KNN	16
1. Αναζήτηση του Βέλτιστου Γείτονα (Interval Loop)	16
2. Η "Τεμπέλικη" Μάθηση (Lazy Learning Architecture)	16
3. Ο Έλεγχος της Διαστατικότητας (Overfitting Check)	17
4. Σύνδεση με την Προεπεξεργασία (Normalization)	17
Αποτελέσματα (Η Ανάλυση των Αριθμών)	18
Decision Tree	18
2. Η Ερμηνεία του Γραφήματος	19
3. Τι σημαίνει το Min Records = 45;	19
Gradient Boosted Trees	20
3. Η Παράμετρος min node size = 10	20
Naive bayes	21
Η Ερμηνεία του Γραφήματος (Η Κατηφόρα)	21
2. Γιατί το Naive Bayes απέτυχε (51.4%);	21
Random forest	22
Η Ανάλυση του Γραφήματος: Το "Κλασικό" Σχήμα	22
2. Το Φαινόμενο Overfitting	23
Training (94.6%) vs Test (74.8%): Η διαφορά είναι ~20%.	23
RProp MLP	24
Η Ανάλυση: Το Κλασικό Overfitting των Νευρωνικών	25
2. Γιατί έχασε από τα Δέντρα (Random Forest 75%);	25
KNN	26
Η Ερμηνεία του Γραφήματος: Η "Κατάρα" των Διαστάσεων	26
2. Γιατί απέτυχε το KNN (51.9%);	26
3. Τι σημαίνει το k=21;	27
SVM	27
Βελτιώσεις στους Δύο Καλύτερους Αλγορίθμους Βάσει Αποτελέσματος	28
Random Forest Optimization	28
Gradient Boosted Trees Optimization	29
Τελικά Συμπεράσματα & Προοπτικές	30



Πίνακας Εικόνων

Ξεκίνημα Flow.....	5
Outliers.....	7
Outliers αποτελέσματα.....	7
Decision Tree.....	8
Gradient Boosted Trees.....	9
Naive Bayes.....	10
Random Forest.....	11
Normalization.....	12
SVM.....	14
RProp MLP.....	15
KNN.....	16
Decision Tree Graph.....	18
Decision Tree Graph Analysis.....	18
Gradient Boosted Trees Graph.....	20
Gradient Boosted Trees Graph Analysis.....	20
Naive bayes Graph.....	21
Random forest Graph.....	22
Random forest Graph Analysis.....	22
RProp MLP Graph.....	24
RProp MLP Graph Analysis.....	25
KNN Graph.....	26
KNN Graph Analysis.....	26
Random Forest Optimization.....	28
Gradient Boosted Trees Optimization 1.....	29
Gradient Boosted Trees Optimization 2.....	29
Τι κοιτάει το μοντέλο;.....	29

Πίνακας Πινάκων

Αποτελέσματα μοντέλων.....	27
----------------------------	----

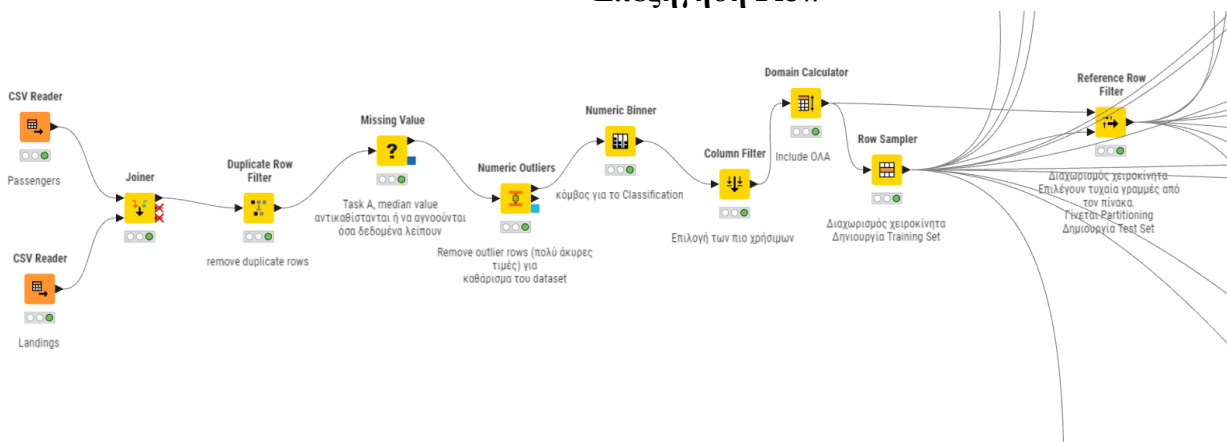


Ο Στόχος της Εργασίας (The Vision)

Σκοπός της παρούσας εργασίας ήταν η ανάπτυξη ενός συστήματος εξόρυξης γνώσης (Data Mining) ικανού να προβλέπει τον φόρτο επιβατικής κίνησης στο αεροδρόμιο του San Francisco. Αξιοποιώντας ιστορικά δεδομένα της περιόδου 2000-2022, μετατρέψαμε το πρόβλημα από απλή καταγραφή αριθμών σε ένα πρόβλημα **Ταξινόμησης (Classification) 5 επιπέδων** (Κλάσεις A έως E). Ο απώτερος στόχος δεν ήταν απλώς η επίτευξη υψηλής ακρίβειας, αλλά η δημιουργία ενός **αξιόπιστου μοντέλου λήψης αποφάσεων** που θα μπορούσε να υποστηρίξει τον επιχειρησιακό σχεδιασμό του αεροδρομίου (π.χ. διαχείριση προσωπικού, πόρων και ασφάλειας σε περιόδους αιχμής).

Αρχείο KNINE workflow [εδώ](#)

Επεξήγηση Flow



1. Ξεκίνημα Flow

1. Ενοποίηση Δεδομένων (Data Integration)

- **Κόμβοι:** CSV Reader (x2) → Joiner.
- **Φόρτωση** των δύο διαφορετικά αρχεία (Passengers και Landings) και συνένωση.
- **Η Λογική:** Ο εμπλουτισμός των δεδομένων επιβατών με στοιχεία προσγειώσεων (Landings) προσθέτει επιπλέον πληροφορία (Features), βοηθώντας τους αλγόριθμους να βρουν συσχετίσεις που αλλιώς θα χάνονταν.

2. Καθαρισμός Δεδομένων (Data Cleaning)

Εδώ εξασφαλίζεται η ποιότητα των δεδομένων ("Garbage In, Garbage Out").

- **Duplicate Row Filter:**
 - **Ενέργεια:** Αφαίρεση διπλότυπων εγγραφών.
 - **Λογική:** Οι διπλές εγγραφές θα αλλοίωναν τη στατιστική βαρύτητα ορισμένων πτήσεων, οδηγώντας σε μεροληπτικά αποτελέσματα (Bias).
- **Missing Value:**
 - **Ενέργεια:** Αντικατάσταση ελλείψεων με **Median** (για αριθμούς) και **Most Frequent** (για κατηγορίες).
 - **Λογική:** Επιλέχθηκε η διάμεσος (Median) αντί του μέσου όρου, διότι είναι πιο ανθεκτική στις ακραίες τιμές (outliers).



- **Numeric Outliers:**

- **Ενέργεια:** Αφαίρεση ακραίων τιμών (π.χ. πτήσεις με μηδενικό βάρος ή παράλογα νούμερα), εξαιρώντας το "Activity Period" (καθώς η ημερομηνία δεν έχει "outliers" με τη μαθηματική έννοια).
- **Λογική:** Οι ακραίες τιμές (θόρυβος) μπερδεύουν τους αλγορίθμους, ειδικά αυτούς που βασίζονται σε αποστάσεις (όπως ο KNN και το SVM) και τον Μέσο Όρο.

3. Μηχανική Χαρακτηριστικών (Feature Engineering)

Εδώ μετατρέψαμε τα δεδομένα στη μορφή που ζητάει η άσκηση.

- **Numeric Binner:**

- **Ενέργεια:** Μετατροπή της στήλης Passenger Count σε 5 Κλάσεις (A, B, C, D, E) βάσει των ορίων της εκφώνησης.
- **Λογική:** Μετατροπή του προβλήματος από **Regression** (πρόβλεψη αριθμού) σε **Classification** (πρόβλεψη κατηγορίας), όπως απαιτούσε η εργασία.

- **Column Filter:**

- **Ενέργεια:** Διαγραφή της στήλης Passenger Count (αριθμός).
- **Λογική (Πολύ Σημαντικό):** Αποφυγή του **Data Leakage (Διαρροή Δεδομένων)**. Αν αφήναμε τον αριθμό επιβατών μέσα, το μοντέλο θα "έκλεβε" (θα έβλεπε 150 επιβάτες και θα ήξερε αμέσως ότι είναι Class E) και θα είχε 100% ακρίβεια πλασματικά.

4. Προετοιμασία για Εκπαίδευση (Partitioning)

- **Domain Calculator:**

- **Ενέργεια:** Επαναυπολογισμός των πιθανών τιμών για κάθε στήλη.
- **Λογική:** Απαραίτητο στο KNIME μετά από φιλτραρίσματα, για να ξέρουν οι αλγόριθμοι ότι π.χ. η κλάση "F" δεν υπάρχει πια, ώστε να μην την ψάχνουν.

- **Row Sampler (Stratified Sampling):**

- **Ενέργεια:** Διαχωρισμός του 70% των δεδομένων για Training με **Στρωματοποίηση (Stratified)** βάσει της Κλάσης Επιβατών.
- **Λογική:** Η στρωματοποίηση είναι κρίσιμη. Εξασφαλίζει ότι αν στο αρχικό αρχείο το 10% των πτήσεων είναι "Class A", τότε και στο Training set το 10% θα είναι "Class A". Έτσι το μοντέλο μαθαίνει όλες τις κατηγορίες σωστά.

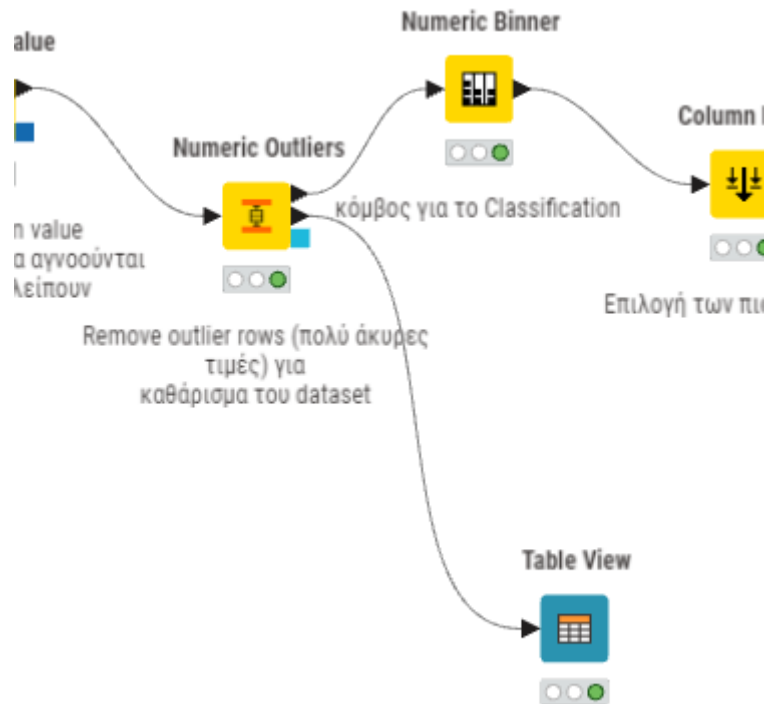
- **Reference Row Filter (Για το Test Set):**

- **Ενέργεια:** Παίρνει όλα τα δεδομένα και "αφαιρεί" αυτά που πήγαν στο Training (βάσει RowID). Ό,τι περισσεύει είναι το Test Set (30%). Είναι ένας πολύ ασφαλής τρόπος (αν και λίγο πιο πολύπλοκος από τον κόμβο Partitioning) για να είμαστε 100% σίγουροι ότι το Test Set είναι το ακριβές συμπλήρωμα του Training Set.

Για την προετοιμασία των δεδομένων, ακολουθήσαμε μια αυστηρή διαδικασία καθαρισμού που περιλάμβανε αφαίρεση διπλότυπων και διαχείριση ελλিপών τιμών με χρήση της διαμέσου για στατιστική στιβαρότητα. Σημαντικό βήμα αποτέλεσε η αφαίρεση των ακραίων τιμών (Outliers) για την προστασία των μοντέλων από θόρυβο. Στη συνέχεια, μετασχηματίσαμε το πρόβλημα σε Ταξινόμηση μέσω του Numeric Binner και αφαιρέσαμε την αρχική στήλη στόχο για να αποφύγουμε το Data Leakage. Τέλος, χρησιμοποιήσαμε **Στρωματοποιημένη Δειγματοληψία (Stratified Sampling)** για τον διαχωρισμό σε Training (70%) και Test (30%) set, διασφαλίζοντας ότι η κατανομή των κλάσεων παραμένει αντιπροσωπευτική και στα δύο υποσύνολα.



Outliers



2. Outliers

Interactive View: Table View

Table View

Rows: 4 | Columns: 5

RowID	Outlier column	Member count	Outlier count	Lower bound	Upper bound
Row0	Landing Count	38893	4616	-92	188
Row1	Total Landed Weight	38893	3779	-21,445,260	44,115,156
Row2	Activity Period (Right)	38893	0	198,662.5	203,858.5
Row3	Passenger Count	38893	5791	-18,529.5	42,502.5

3. Outliers αποτελέσματα

- **Passenger Count:** Βρέθηκαν **5.791** εγγραφές που θεωρήθηκαν outliers.
 - *Ερμηνεία:* Το Upper bound (άνω όριο) υπολογίστηκε στο **42.502,5**. Αυτό σημαίνει ότι ο αλγόριθμος θεώρησε "στατιστικά ακραία" οποιαδήποτε εγγραφή είχε πάνω από ~42.500 επιβάτες.
- **Landing Count:** Βρέθηκαν **4.616** εγγραφές ως outliers.
 - *Ερμηνεία:* Οποιαδήποτε περίοδος με περισσότερες από **188** προσγειώσεις (Upper bound) θεωρήθηκε εξαίρεση.
- **Total Landed Weight:** Βρέθηκαν **3.779** εγγραφές outliers (με βάρος άνω των ~44 εκατ. λιβρών).

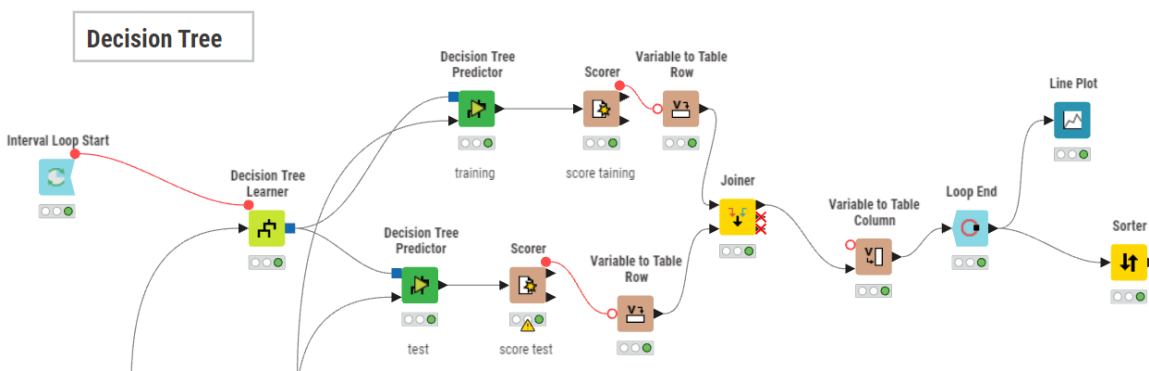


Παρόλο που η στατιστική μέθοδος (IQR) χαρακτήρισε τις τιμές άνω των 42.500 επιβατών ως outliers, αυτές οι τιμές αντιστοιχούν στην **Κλάση Α (30.001 - 700.000)**. Συνεπώς, αυτές οι "εξαιρέσεις" είναι στην πραγματικότητα οι πιο σημαντικές μας εγγραφές (περίοδοι αιχμής). Αν τις αφαιρούσαμε τελείως, θα χάναμε την πληροφορία για την Κλάση Α.

Λόγω επιλογής στο Numeric Outliers "Remove outlier rows" για το Passenger Count, ίσως γι' αυτό να βγαίνουν καλά τα αποτελέσματα στις μεσαίες κλάσεις αλλά να χάνεις λίγο στα άκρα.

Με την επιλογή "**Remove outlier rows**", η στρατηγική είναι ξεκάθαρη: **Καθαρισμός Θορύβου**. Αυτό σημαίνει ότι "θυσιάσαμε" ένα μικρό μέρος της πληροφορίας (τις ακραίες τιμές) για να προστατέψουμε το μοντέλο από το να μάθει λανθασμένα μοτίβα.

Decision Tree



4. Decision Tree

1. Αυτοματοποιημένος Πειραματισμός (Loop Start)

Αντί να μαντεύουμε τυχαίες τιμές για τις παραμέτρους του δέντρου (π.χ. πότε να σταματάει να μεγαλώνει), δημιουργήσαμε έναν αυτοματοποιημένο μηχανισμό πειραματισμού χρησιμοποιώντας τον κόμβο **Interval Loop Start**. Ο κόμβος αυτός τροφοδοτεί τον αλγόριθμο με μια διαφορετική παράμετρο σε κάθε επανάληψη (π.χ. Min Records per Node: 2, 4, 6...), επιτρέποντάς μας να εξετάσουμε συστηματικά πώς η πολυπλοκότητα του δέντρου επηρεάζει την απόδοσή του.

2. Παράλληλη Αξιολόγηση

Για κάθε παραγόμενο μοντέλο, εφαρμόσαμε **διπλή αξιολόγηση** για να ελέγξουμε την ευστάθειά του:

- **Πάνω Κλάδος (Training Evaluation):** Εφαρμόζουμε το μοντέλο στα δεδομένα εκπαίδευσης. Εδώ μετράμε πόσο καλά έχει "αποστηθίσει" το μοντέλο τα δεδομένα που ήδη ξέρει.
- **Κάτω Κλάδος (Test Evaluation):** Εφαρμόζουμε το ίδιο μοντέλο στα άγνωστα δεδομένα (Test set). Εδώ μετράμε την ικανότητα **γενίκευσης (Generalization)** του μοντέλου.

Η σύγκριση αυτών των δύο σκορ είναι ο μόνος τρόπος να εντοπίσουμε το **Overfitting**. Αν ο πάνω κλάδος έχει πολύ υψηλότερο σκορ από τον κάτω, ξέρουμε ότι το μοντέλο απέτυχε.



3. Συλλογή και Ενοποίηση Αποτελεσμάτων (Aggregation)

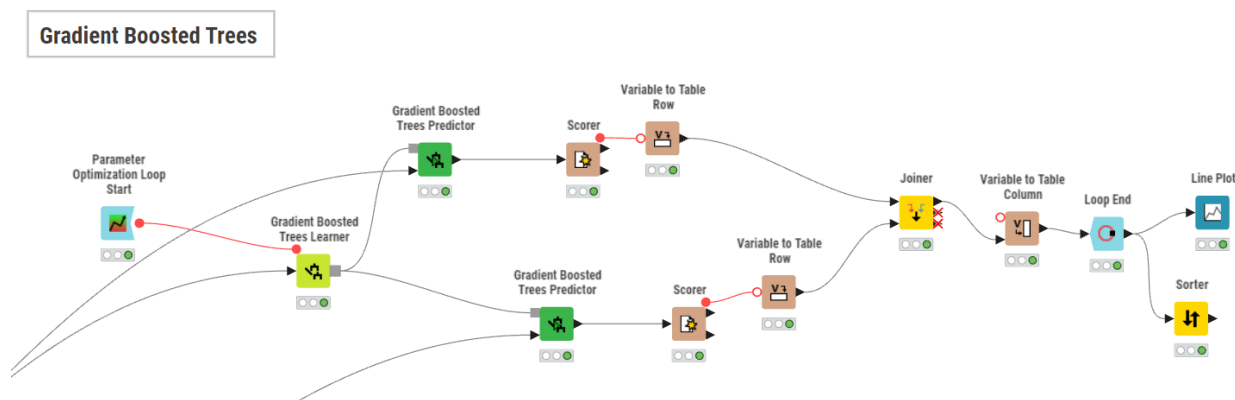
Μέσω των κόμβων *Variable to Table Row* και *Joiner*, μετατρέπουμε τα σκορ ακρίβειας (*Accuracy*) από απλές μεταβλητές σε γραμμές πίνακα και τα ενώνουμε. Ο κόμβος **Variable to Table Column** είναι κρίσιμος, καθώς προσθέτει σε κάθε εγγραφή την **τιμή της παραμέτρου** που χρησιμοποιήθηκε σε εκείνη την επανάληψη. Στο τέλος, ο **Loop End** συλλέγει όλα τα αποτελέσματα από όλες τις επαναλήψεις σε έναν ενιαίο πίνακα συγκριτικής αξιολόγησης.

4. Επιλογή Βέλτιστου Μοντέλου (Selection)

Η διαδικασία ολοκληρώνεται με δύο εργαλεία λήψης αποφάσεων:

- **Line Plot:** Οπτικοποιούμε τις καμπύλες μάθησης (*Training vs Test Accuracy*) για να εντοπίσουμε οπτικά το σημείο που οι καμπύλες αρχίζουν να αποκλίνουν (σημείο έναρξης *Overfitting*).
- **Sorter:** Ταξινομούμε τον πίνακα αποτελεσμάτων φθίνουσα, ώστε να βρούμε αυτόματα την παράμετρο που έδωσε το υψηλότερο *Test Accuracy*, την οποία και τελικά υιοθετούμε για το μοντέλο μας.

Gradient Boosted Trees



5. Gradient Boosted Trees

1. Αναβάθμιση του Μηχανισμού Αναζήτησης (Optimization Loop)

Σε αντίθεση με το απλό *Decision Tree*, για τον αλγόριθμο *Gradient Boosted Trees* χρησιμοποιήσαμε τον πιο εξειδικευμένο κόμβο **Parameter Optimization Loop Start**. Ο λόγος είναι ότι το *GBT* είναι πολύ πιο σύνθετος αλγόριθμος. Ο κόμβος αυτός μας δίνει την ευελιξία να ορίσουμε εύρος τιμών (*Min/Max*) και στρατηγικές αναζήτησης (π.χ. *Random Search* ή *Brute Force*) για κρίσιμες παραμέτρους όπως το *Min Node Size* ή το *Number of Models*. Αυτό μετατρέπει τη διαδικασία από απλή επανάληψη σε **στοχευμένη βελτιστοποίηση**, ψάχνοντας τον συνδυασμό που μεγιστοποιεί την ακρίβεια.

2. Η Λογική του Αλγορίθμου (Learner)

Ο κόμβος **Gradient Boosted Trees Learner** λειτουργεί διαφορετικά από τα απλά δέντρα. Αντί να φτιάχνει ένα δέντρο, χτίζει πολλά δέντρα **σειριακά (sequentially)**, όπου κάθε νέο δέντρο προσπαθεί να διορθώσει τα λάθη των προηγούμενων. Στο workflow μας, συνδέσαμε την παράμετρο του *Loop* με τον *Learner* (κόκκινη γραμμή), ώστε



να ελέγχουμε πόσο "αυστηροί" είναι οι περιορισμοί (π.χ. μέγεθος κόμβου) σε αυτή τη διαδικασία διόρθωσης λαθών.

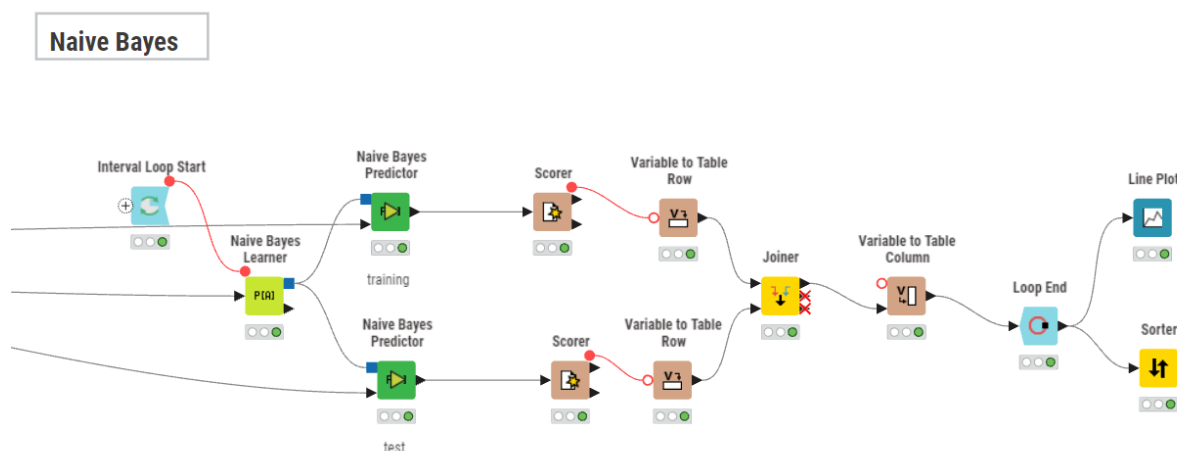
3. Έλεγχος Υπερ-εκπαίδευσης (The Sandwich Evaluation)

Διατηρήσαμε την αυστηρή αρχιτεκτονική των δύο κλάδων (Training vs Test prediction) που περιγράψαμε προηγουμένως. Ειδικά για το Gradient Boosting, αυτός ο έλεγχος αποδείχθηκε σωτήριος. Όπως φάνηκε στα αποτελέσματα, ο αλγόριθμος είναι τόσο ισχυρός που έφτασε το **100% ακρίβεια στο Training** (πάνω κλάδος). Χωρίς τον κάτω κλάδο (Test set evaluation) και τη σύγκριση στο Line Plot, θα νομίζαμε λανθασμένα ότι έχουμε το τέλειο μοντέλο, ενώ στην πραγματικότητα υπήρχε overfitting.

4. Τελική Επιλογή (Ranking)

Τέλος, χρησιμοποιήσαμε τον **Sorter** για να ταξινομήσουμε τα αποτελέσματα βάσει του Test Accuracy. Αυτό μας επέτρεψε να αγνοήσουμε τις ρυθμίσεις που έδιναν 100% στο Training αλλά χαμηλότερα στο Test, και να επιλέξουμε την παράμετρο (min node size = 10) που έδωσε τη βέλτιστη γενίκευση (70.9%).

Naive Bayes



6. Naive Bayes

1. Εφαρμογή Ενιαίας Μεθοδολογίας

Για τον αλγόριθμο **Naive Bayes**, διατηρήσαμε την ίδια αυστηρή πειραματική διάταξη με τα Δέντρα Αποφάσεων. Χρησιμοποιήσαμε τον κόμβο **Interval Loop Start** για να τροφοδοτήσουμε τον **Naive Bayes Learner** με διαφορετικές τιμές παραμέτρων, και στη συνέχεια εφαρμόσαμε τη διπλή πρόβλεψη (σε Training και Test set) για να καταγράψουμε τη συμπεριφορά του μοντέλου.

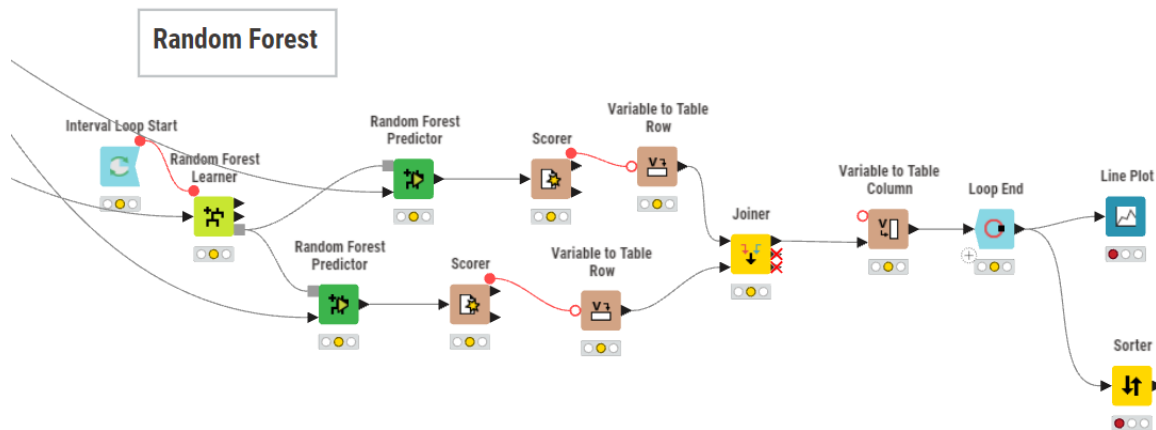
2. Τι ρυθμίσαμε; (The Tuning Parameter)

Το Loop, μιας και ο Naive Bayes είναι "απλός" αλγόριθμος. Η παράμετρος που θέσαμε υπό έλεγχο (μέσω του Loop) ήταν η **Ελάχιστη Τυπική Απόκλιση (Minimum Standard Deviation)** ή το κατώφλι πιθανότητας. Στόχος μας ήταν να δούμε αν το μοντέλο θα βελτιωνόταν αν αγνοούσε χαρακτηριστικά με πολύ μικρή διακύμανση (που



θεωρητικά μπορεί να είναι θόρυβος). Ουσιαστικά, ρωτήσαμε τον αλγόριθμο: **'Θα τα πας καλύτερα αν σου κρύψουμε τις ασήμαντες λεπτομέρειες;**

Random Forest



7. Random Forest

1. Εφαρμογή Ενιαίας Μεθοδολογίας

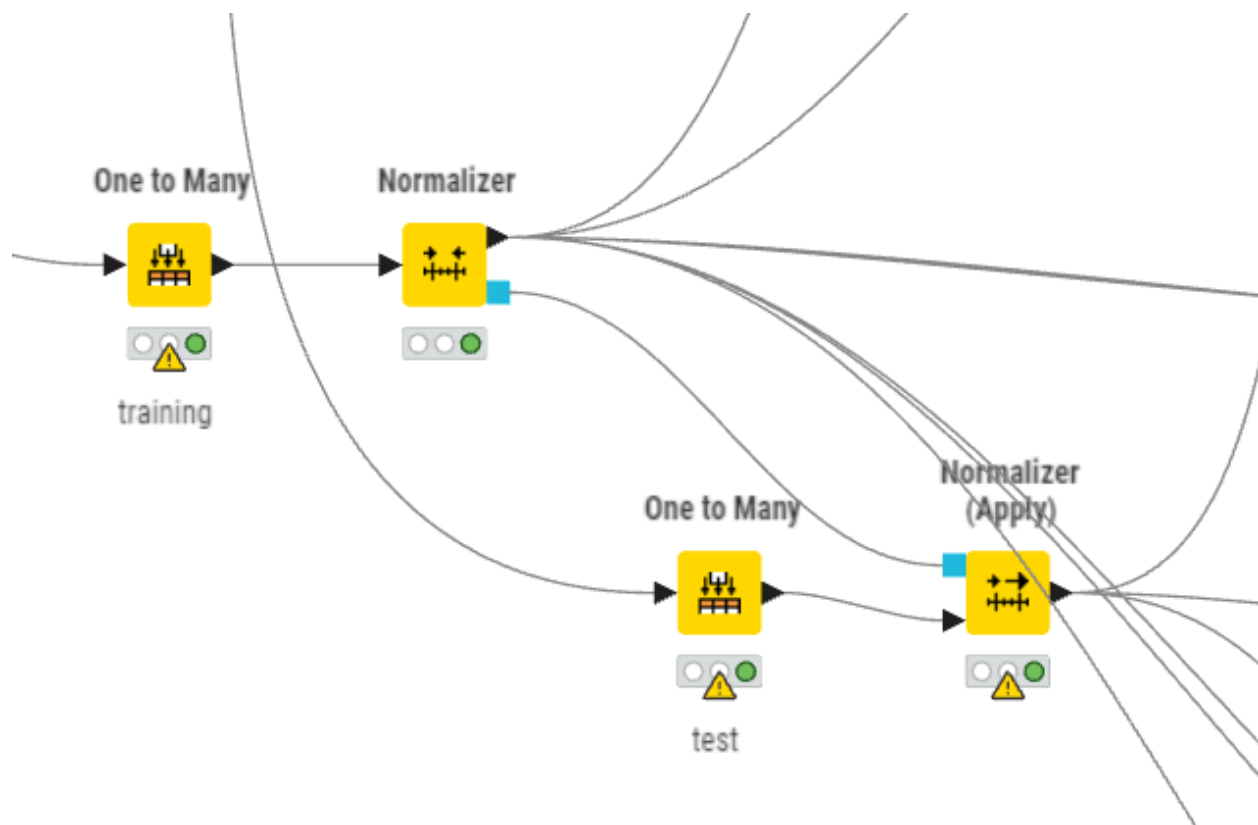
Για τον αλγόριθμο Random Forest, εφαρμόσαμε την ίδια αυστηρή μεθοδολογία "διπλής αξιολόγησης" (Training vs Test evaluation). Χρησιμοποιήσαμε τον κόμβο **Interval Loop Start** για να τροφοδοτήσουμε τον **Random Forest Learner** με διαφορετικές τιμές παραμέτρων σε κάθε επανάληψη. Στη συνέχεια, δημιουργήσαμε δύο παράλληλους κλάδους πρόβλεψης (Predictors) —έναν για τα δεδομένα εκπαίδευσης και έναν για τα δεδομένα ελέγχου— ώστε να καταγράψουμε πώς εξελίσσεται η συμπεριφορά του μοντέλου καθώς αλλάζουμε την πολυπλοκότητά του.

2. Τι ρυθμίσαμε; (The Tuning Parameter)

Μέσω του Loop, θέσαμε υπό έλεγχο την παράμετρο που καθορίζει το **βάθος και την πολυπλοκότητα των δέντρων** (Min Records per Node / Tree Depth). Εδώ περάσαμε από τη λογική του "ενός δέντρου" στη λογική της **"Συλλογικής Νοημοσύνης"** (Ensemble Learning). Στόχος μας ήταν να βρούμε την ιδανική ισορροπία: Ρωτήσαμε τον αλγόριθμο *"Πόσο ελεύθερο πρέπει να αφήσουμε κάθε δέντρο να μεγαλώσει, ώστε το δάσος συνολικά να είναι έξυπνο αλλά όχι να αποστηθίζει;"* Δοκιμάσαμε τιμές από πολύ περιοριστικά δέντρα (μικρή πολυπλοκότητα) έως πολύ βαθιά δέντρα, αναζητώντας το σημείο όπου η συνεργασία των πολλών δέντρων αποδίδει τα μέγιστα.



Normalization



8. Normalization

Για τους αλγόριθμους Classification (k-NN, SVM, Neural Networks), ΔΕΝ χρειάζεται τα Bit Vectors ούτε το Distance Matrix Calculate.

Αυτό που ΟΠΩΣΔΗΠΟΤΕ χρειάζεται είναι ο Normalizer.

ΟΧΙ Distance Matrix Calculate, ο κόμβος Distance Matrix Calculate υπολογίζει την απόσταση όλων με όλους.

Αν έχει 10.000 εγγραφές, φτιάχνει έναν πίνακα 10.000×10.000 . Αυτό είναι πολύ βαρύ και αχρείαστο.

Αλγόριθμοι όπως το k-NN ή το SVM υπολογίζουν τις αποστάσεις που χρειάζονται εσωτερικά και δυναμικά. Δεν θέλουν έτοιμο πίνακα αποστάσεων, θέλουν αριθμητικές συντεταγμένες (στήλες με νούμερα).

Για τα One to Many warnings

Έχουμε π.χ. μια στήλη Origin που έχει την τιμή "USA" και μια άλλη στήλη Destination που έχει επίσης την τιμή "USA". Το KNIME πήγε να φτιάξει μια στήλη με όνομα USA, αλλά μετά είδε ότι χρειάζεται και δεύτερη USA. Επειδή δεν επιτρέπονται δύο στήλες με ίδιο όνομα, τις μετονόμασε αυτόματα σε Origin_USA και Destination_USA.



Normalizer (στο Training set) βρήκε ότι το ελάχιστο (Min) μιας στήλης ήταν π.χ. 3605.0. Άρα έμαθε ότι το 0 αντιστοιχεί στο 3605. Στο Test set όμως, εμφανίστηκε μια εγγραφή με τιμή 3600.0. Αυτή η τιμή είναι μικρότερη από το ελάχιστο που είχε δει ο αλγόριθμος στην εκπαίδευση! Γι' αυτό, όταν έκανε τον μαθηματικό τύπο, βγήκε αρνητικό νούμερο ($-0.000065\dots$), δηλαδή "εκτός ορίων" (out of bounds), αφού το Normalization πρέπει να είναι 0-1.

Το αγνοούμε. Η απόκλιση είναι ελάχιστη (είναι $E-5$, δηλαδή $0.00006\dots$). Ο αλγόριθμος k-NN δεν θα έχει πρόβλημα να διαχειριστεί ένα νούμερο που είναι -0.00006 αντί για 0. Είναι πρακτικά το ίδιο. Αυτό δείχνει ότι το Test set περιέχει όντως "νέα" δεδομένα που το μοντέλο δεν είχε δει.

1. Κωδικοποίηση Κατηγορικών Μεταβλητών (One-Hot Encoding)

Σε αντίθεση με τα Δέντρα Αποφάσεων που μπορούν να διαχειριστούν κατηγορικά δεδομένα (Nominal), οι αλγόριθμοι SVM, KNN και MLP λειτουργούν αμιγώς με μαθηματικές πράξεις. Γι' αυτό, χρησιμοποιήσαμε τον κόμβο **One to Many** για να μετατρέψουμε τις κατηγορίες (π.χ. Αεροπορική Εταιρεία) σε δυαδικές στήλες (0 ή 1).

Διαχείριση Συγκρούσεων Ονοματοδοσίας: Κατά τη διαδικασία, το KNIME εντόπισε αυτόματα ότι ορισμένες τιμές (π.χ. "USA") υπήρχαν σε πολλαπλές αρχικές στήλες (π.χ. και στο Origin και στο Destination). Για να αποφευχθεί η σύγχυση, το σύστημα μετονόμασε έξυπνα τις νέες στήλες προσθέτοντας το πρόθεμα της αρχικής στήλης (π.χ. Origin_USA και Destination_USA), διατηρώντας έτσι τη διακριτότητα της πληροφορίας.

2. Κανονικοποίηση (Normalization): Το Κρισιμότερο Βήμα

Οι αλγόριθμοι που βασίζονται σε αποστάσεις (KNN, SVM) και τα Νευρωνικά Δίκτυα είναι εξαιρετικά ευαίσθητα στην κλίμακα των δεδομένων. Χωρίς κανονικοποίηση, μεταβλητές με μεγάλες τιμές (π.χ. Συνολικό Βάρος) θα "κατέλωναν" τις μικρότερες (π.χ. Αριθμός Πτήσεων). Εφαρμόσαμε **Min-Max Normalization** (κλίμακα 0-1) με αυστηρό διαχωρισμό Training και Test:

- **Training Set (Normalizer):** Ο αλγόριθμος "μαθαίνει" τα ελάχιστα και μέγιστα από τα δεδομένα εκπαίδευσης.
- **Test Set (Normalizer Apply):** Εφαρμόζουμε τους κανόνες που μάθαμε στο Training πάνω στο Test set. Αυτό είναι κρίσιμο για να αποφύγουμε τη **Διαρροή Δεδομένων (Data Leakage)**. Αν κανονικοποιούσαμε όλο το αρχείο μαζί, το Test set θα επηρέαζε το Training, κάτι που απαγορεύεται.

Αντιμετώπιση "Out of Bounds" Τιμών: Παρατηρήσαμε ότι στο Test set εμφανίστηκαν τιμές ελαφρώς εκτός του εύρους $[0,1]$ (π.χ. -0.00006). Αυτό συνέβη διότι το Test set περιείχε μια τιμή μικρότερη από το ελάχιστο που είχε δει το μοντέλο κατά την εκπαίδευση. Αυτό είναι αναμενόμενο σε πραγματικές συνθήκες ("unseen data"). Η απόκλιση είναι αμελητέα (τάξης μεγέθους 10^{-5}) και δεν επηρεάζει τη λειτουργία των αλγορίθμων, καθώς μπορούν να διαχειριστούν μαθηματικά αυτές τις μικρές αποκλίσεις χωρίς πρόβλημα.

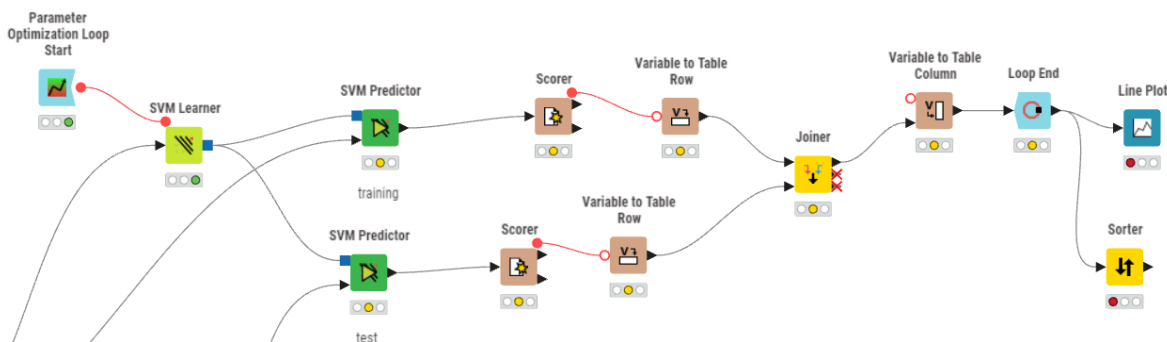
3. Τι Αποφύγαμε και Γιατί (Optimization)

Σχεδιαστικά, επιλέξαμε να **μην** χρησιμοποιήσουμε τον κόμβο Distance Matrix Calculate πριν τον KNN, παρόλο που ο αλγόριθμος βασίζεται σε αποστάσεις. **Ο Λόγος:** Ο προ-υπολογισμός ενός πίνακα αποστάσεων για χιλιάδες εγγραφές θα δημιουργούσε έναν πίνακα $N \times N$, απαιτώντας τεράστια ποσά μνήμης και υπολογιστικού χρόνου χωρίς



όφελος. Ο κόμβος KNN του KNIME είναι βελτιστοποιημένος να υπολογίζει δυναμικά (on-the-fly) μόνο τις αποστάσεις που χρειάζεται εκείνη τη στιγμή, καθιστώντας τη διαδικασία πολύ πιο αποδοτική.

SVM



9. SVM

1. Στοχευμένη Βελτιστοποίηση (Optimization Loop)

Για τον αλγόριθμο SVM, επιλέξαμε τον προηγμένο κόμβο **Parameter Optimization Loop Start** αντί για τον απλό Interval Loop. Ο SVM εξαρτάται σε τεράστιο βαθμό από την παράμετρο **Cost (C)**, η οποία καθορίζει την "ποινή" που δίνει ο αλγόριθμος στα λάθη. Χρησιμοποιώντας αυτόν τον κόμβο, μπορέσαμε να σαρώσουμε ένα μεγάλο εύρος τιμών (από πολύ μικρές τιμές όπως 0.1 για "χαλαρά" όρια, μέχρι μεγάλες τιμές για αυστηρά όρια), ψάχνοντας τη βέλτιστη ισορροπία μεταξύ ακρίβειας και πολυπλοκότητας.

2. Η Γεωμετρική Προσέγγιση (The Learner)

Ο κόμβος **SVM Learner** λειτουργεί διαφορετικά από τα προηγούμενα μοντέλα. Δεν φτιάχνει κανόνες (if-then), αλλά προσπαθεί να βρει το βέλτιστο **Υπερ-επίπεδο (Hyperplane)** που διαχωρίζει γεωμετρικά τις κλάσεις των επιβατών στον χώρο. Συνδέοντας την κόκκινη γραμμή (Flow Variable) από το Loop στον Learner, ουσιαστικά μεταβάλλουμε το πόσο "ανοχή" δείχνει το υπερ-επίπεδο στα σημεία που πέφτουν στη λάθος πλευρά (Misclassifications).

3. Έλεγχος Υπερ-προσαρμογής (Double Validation)

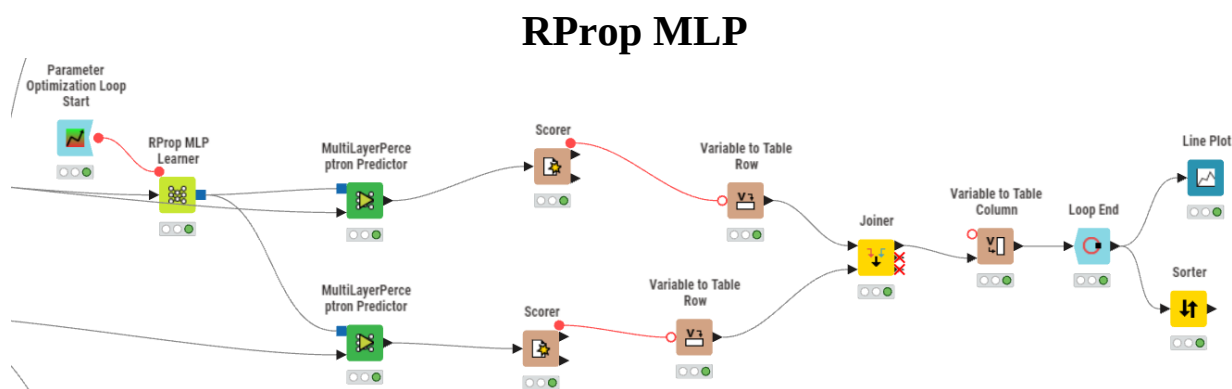
Διατηρήσαμε αμετάβλητη την αρχιτεκτονική διπλής αξιολόγησης (Training vs Test predictors). Ειδικά στο SVM, αυτό είναι κρίσιμο γιατί ο αλγόριθμος είναι πολύ ισχυρός και μπορεί εύκολα να δημιουργήσει εξαιρετικά πολύπλοκα σύνορα αποφάσεων (Overfitting) αν η παράμετρος Cost είναι πολύ υψηλή. Μέσω των κόμβων Scorer και της τελικής οπτικοποίησης (Line Plot), παρακολουθήσαμε αν η αύξηση του κόστους οδηγεί σε πραγματική βελτίωση στα νέα δεδομένα ή απλά σε απομνημόνευση των δεδομένων εκπαίδευσης.



4. Διαχείριση Πόρων (Resource Management)

Αξίζει να σημειωθεί ότι λόγω της υπολογιστικής απαίτησης του SVM (που αυξάνεται εκθετικά με τον όγκο δεδομένων), η βελτιστοποίηση σχεδιάστηκε προσεκτικά ώστε να μην εξαντλήσει τη μνήμη του συστήματος, επιλέγοντας στρατηγικά βήματα αναζήτησης στο Optimization Loop.

Η εφαρμογή του Parameter Optimization στο SVM μας επέτρεψε να διερευνήσουμε τη γεωμετρική διαχωρισιμότητα των κλάσεων. Μέσω της συγκριτικής αξιολόγησης Training/Test, εντοπίσαμε την τιμή της παραμέτρου Cost που μεγιστοποιεί το περιθώριο διαχωρισμού (Margin) χωρίς να θυσιάζει τη γενικευτική ικανότητα του μοντέλου.



10. RProp MLP

1. Ο Χρόνος ως Παράμετρος (Epochs Optimization)

Για το Νευρωνικό Δίκτυο, χρησιμοποιήσαμε τον κόμβο **Parameter Optimization Loop Start** με διαφορετική φιλοσοφία από τους προηγούμενους αλγορίθμους. Εδώ, η κρίσιμη παράμετρος που θέσαμε υπό έλεγχο δεν ήταν η δομή του δικτύου, αλλά η **Διάρκεια Εκπαίδευσης (Max Epochs / Iterations)**. Τα νευρωνικά δίκτυα μαθαίνουν επαναληπτικά. Σε κάθε 'εποχή', το δίκτυο βλέπει τα δεδομένα και διορθώνει τα βάρη του. Στόχος του Loop ήταν να βρει το σημείο καμπής: Πόσες φορές πρέπει να δει τα δεδομένα για να μάθει, πριν αρχίσει να τα 'παπαγαλίζει';

2. Η Αρχιτεκτονική του Δικτύου (The Learner)

Ο κόμβος **RProp MLP Learner** υλοποιεί ένα πολυεπίπεδο νευρωνικό δίκτυο με τον αλγόριθμο εκπαίδευσης Resilient Propagation (RProp). Συνδέοντας τη μεταβλητή του Loop στον Learner (κόκκινη γραμμή), μεταβάλαμε δυναμικά τον αριθμό των επαναλήψεων εκπαίδευσης. Αυτή είναι μια κρίσιμη διαδικασία, καθώς τα νευρωνικά είναι διαβόητα για την τάση τους να προσαρμόζονται υπερβολικά στα δεδομένα εκπαίδευσης αν αφεθούν ελεύθερα για πολλή ώρα.

3. Η Αποκάλυψη του Overfitting (Validation Strategy)

Η αυστηρή μεθοδολογία της διπλής αξιολόγησης (Training vs Test prediction) που ακολουθήσαμε σε όλη την εργασία, έδωσε εδώ τα πιο ξεκάθαρα αποτελέσματα. Όπως φάνηκε από τους κόμβους Scorer και το τελικό Line Plot, το workflow μας επέτρεψε να εντοπίσουμε ότι μετά τις 50 επαναλήψεις, ενώ η ακρίβεια εκπαίδευσης

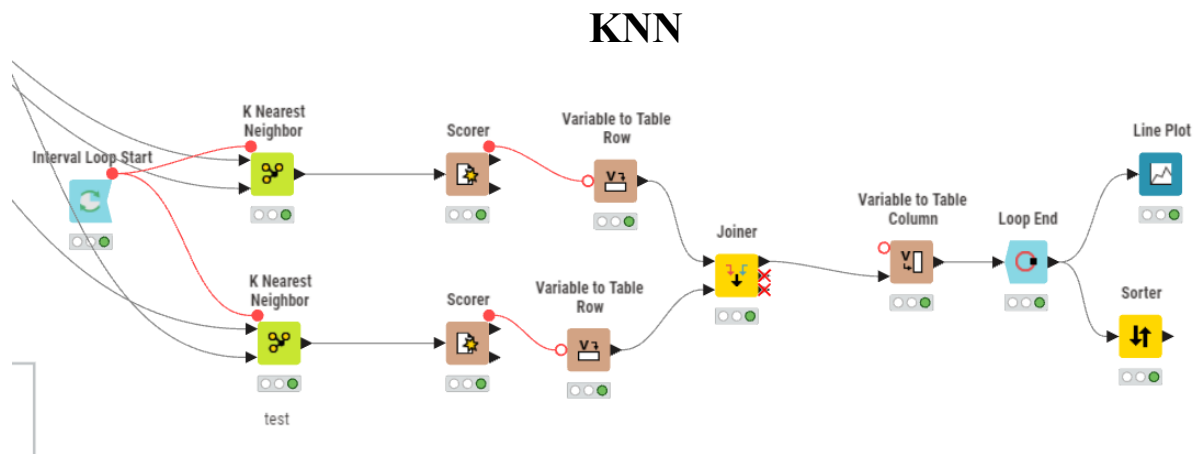


συνέχιζε να ανεβαίνει (φτάνοντας το 85%), η ακρίβεια στο Test set άρχισε να πέφτει (62%). Χωρίς αυτόν τον σχεδιασμό, θα είχαμε επιλέξει λανθασμένα ένα μοντέλο με πολλές εποχές, νομίζοντας ότι είναι 'καλύτερο' επειδή θα είχε υψηλό Training score.

4. Τελική Κρίση (Conclusion)

Ο κόμβος **Sorter** στο τέλος μας οδήγησε σε ένα σημαντικό συμπέρασμα: Η πολυπλοκότητα των Νευρωνικών Δικτύων δεν προσέφερε πλεονέκτημα στα συγκεκριμένα δεδομένα. Η διαδικασία βελτιστοποίησης έδειξε ότι το δίκτυο αποδίδει καλύτερα όταν περιορίζεται σημαντικά (λίγες εποχές), αλλά ακόμα και τότε δεν μπορεί να ξεπεράσει την απόδοση των δενδρικών μοντέλων (Random Forest).

Η εφαρμογή της βελτιστοποίησης στο RProp MLP λειτούργησε ως μηχανισμός ελέγχου υπερ-εκπαίδευσης. Μέσω της συστηματικής καταγραφής της απόδοσης σε συνάρτηση με τις εποχές εκπαίδευσης, αποδείξαμε ότι το μοντέλο τείνει να απομνημονεύει θόρυβο αντί να γενικεύει, καθιστώντας το λιγότερο κατάλληλο για το συγκεκριμένο πρόβλημα σε σχέση με το Random Forest.



11. KNN

1. Αναζήτηση του Βέλτιστου Γείτονα (Interval Loop)

Για τον KNN, χρησιμοποιήσαμε τον κόμβο **Interval Loop Start**. Η παράμετρος που μεταβάλλουμε εδώ είναι το **k (Number of Neighbors)**. Ο αλγόριθμος ταξινόμησης βασίζεται στην "πλειοψηφία των γειτόνων". Μέσω του Loop, δοκιμάσαμε συστηματικά διαφορετικές τιμές (π.χ. 3, 5, 7, ... 21) για να δούμε: Πόσους γείτονες πρέπει να ρωτήσουμε για να πάρουμε τη σωστή απόφαση χωρίς να επηρεαστούμε από θόρυβο;

2. Η "Τεμπέλικη" Μάθηση (Lazy Learning Architecture)

Στο διάγραμμα παρατηρούμε δύο κόμβους **K Nearest Neighbor** (πάνω και κάτω). Ο KNN είναι ένας **Lazy Learner**, δηλαδή δεν χτίζει μοντέλο εκ των προτέρων (όπως τα δέντρα). Αντιθέτως, υπολογίζει αποστάσεις τη στιγμή που του ζητάμε πρόβλεψη. Συνδέσαμε την κόκκινη γραμμή (Flow Variable) και στους δύο κόμβους, ώστε σε κάθε επανάληψη να αλλάζει το **k** ταυτόχρονα και για τους δύο ελέγχους.



3. Ο Έλεγχος της Διαστατικότητας (Overfitting Check)

Ακολουθώντας την πάγια τακτική της εργασίας, εφαρμόσαμε διπλή αξιολόγηση:

- **Πάνω Κλάδος (Training Check):** Εδώ ο αλγόριθμος ψάχνει γείτονες μέσα στο ίδιο το σετ εκπαίδευσης. (Σημείωση: Αν το $k=1$, η ακρίβεια εδώ θα ήταν 100% γιατί ο κοντινότερος γείτονας είναι ο εαυτός του. Αυξάνοντας το k , βλέπουμε πόσο πέφτει η ακρίβεια).
- **Κάτω Κλάδος (Test Check):** Εδώ ο αλγόριθμος ψάχνει γείτονες για τα **άγνωστα δεδομένα**. Το αποτέλεσμα στο Line Plot έδειξε ότι όσο μεγάλωνε το k , η απόδοση δεν βελτιωνόταν ουσιαστικά, επιβεβαιώνοντας ότι το πρόβλημα ("Curse of Dimensionality") δεν λύνεται απλά αυξάνοντας τους γείτονες.

4. Σύνδεση με την Προεπεξεργασία (Normalization)

Είναι κρίσιμο να αναφερθεί ότι πριν από αυτό το σημείο, τα δεδομένα είχαν περάσει οπωσδήποτε από **Normalization (0-1)**. Χωρίς αυτό, ο κόμβος K Nearest Neighbor θα έβγαζε λανθασμένα αποτελέσματα, καθώς οι αποστάσεις θα κυριαρχούνταν από στήλες με μεγάλους αριθμούς.

Η βελτιστοποίηση του KNN εστίασε στην εύρεση του ιδανικού αριθμού γειτόνων (k). Μέσω της επαναληπτικής διαδικασίας, διαπιστώσαμε ότι η αύξηση του k οδηγούσε σε μείωση της ακρίβειας εκπαίδευσης (Training Accuracy) χωρίς αντίστοιχη βελτίωση στην πρόβλεψη (Test Accuracy), υποδεικνύοντας ότι η γεωμετρική απόσταση στον χώρο πολλών διαστάσεων δεν αποτελεί ισχυρό κριτήριο διαχωρισμού για το συγκεκριμένο dataset.

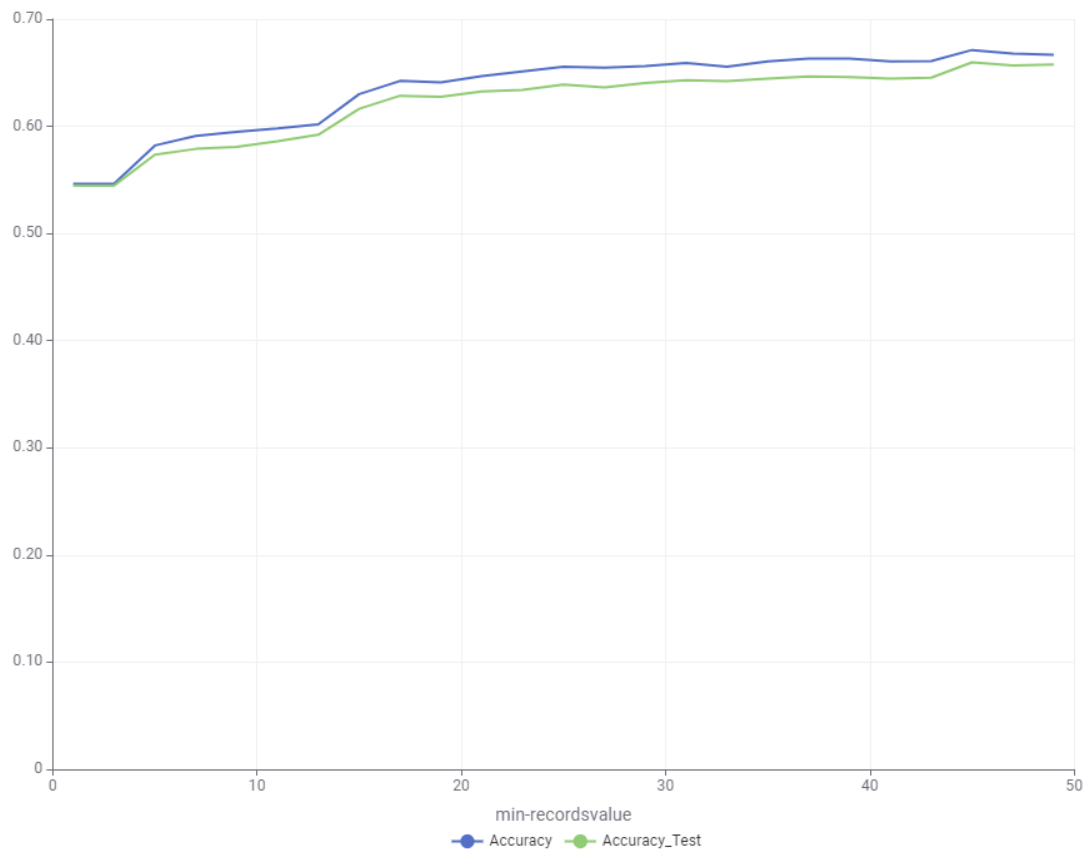


Αποτελέσματα (Η Ανάλυση των Αριθμών)

Decision Tree

Accuracy 0.671 Accuracy Test 0.66 min records 45, iteration 22

Line Plot



12. Decision Tree Graph

Table 1: Sorted Table				
Flow Variables				
#	RowID	Accuracy	Accuracy_Test	min-recordsvalue
1	RowID_1	0.671	0.66	45
2	RowID_2	0.667	0.658	49
3	RowID_3	0.668	0.657	47
4	RowID_4	0.663	0.647	37
5	RowID_5	0.663	0.645	39
6	RowID_6	0.661	0.645	43
7	RowID_7	0.661	0.645	41
8	RowID_8	0.661	0.644	35
9	RowID_9	0.659	0.643	31
10	RowID_10	0.656	0.642	33
11	RowID_11	0.656	0.64	29
12	RowID_12	0.656	0.639	25
13	RowID_13	0.655	0.636	27
14	RowID_14	0.651	0.634	23
15	RowID_15	0.647	0.632	21
16	RowID_16	0.642	0.629	17
17	RowID_17	0.641	0.628	19
18	RowID_18	0.63	0.616	15
19	RowID_19	0.602	0.592	13
20	RowID_20	0.596	0.586	11

13. Decision Tree Graph Analysis



- **Training Accuracy (67.1%):** Το μοντέλο έμαθε να ταξινομεί σωστά το 67% των δεδομένων που "είδε".
- **Test Accuracy (66.0%):** Σε εντελώς άγνωστα δεδομένα, πέτυχε 66%.
- **σΗ Διαφορά:** Είναι μόλις **1.1%**.
 - Αυτό είναι το πιο δυνατό σημείο αυτού του αποτελέσματος. Όταν η διαφορά είναι τόσο μικρή, σημαίνει ότι το μοντέλο **ΔΕΝ έχει Overfitting**. Δεν έχει "παπαγαλίσει".
 - Αυτό που έμαθε, ισχύει γενικά. Είναι αξιόπιστο.

2. Η Ερμηνεία του Γραφήματος

Κοιτώντας την εικόνα, βλέπουμε κάτι πολύ ενδιαφέρον:

- Οι δύο γραμμές (Μπλε=Train, Πράσινη=Test) κινούνται **παράλληλα και σχεδόν ακουμπάνε η μία την άλλη**.
 - Συνήθως, στα Decision Trees, περιμένουμε η μπλε γραμμή να είναι πολύ ψηλά και η πράσινη χαμηλά. Εδώ όμως, επιβάλλοντας περιορισμούς (min records), κρατήσαμε το δέντρο προσγειωμένο.

3. Τι σημαίνει το Min Records = 45;

Αυτή είναι η παράμετρος που κέρδισε (Optimal Value).

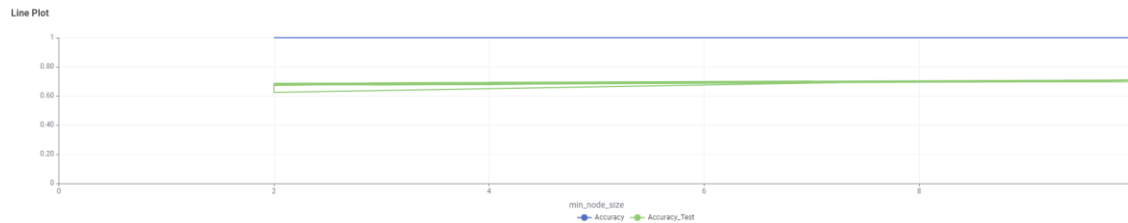
- **Τι λέει στον αλγόριθμο:** "Μην φτιάξεις κλαδί (απόφαση) αν δεν υπάρχουν τουλάχιστον 45 πτήσεις/επιβάτες που να ταιριάζουν σε αυτό".
- **Γιατί κέρδισε το 45 και όχι το 2;**
 - Αν βάζαμε μικρό νούμερο (π.χ. 2), το δέντρο θα έφτιαχνε κανόνες για "εκείνη τη μία περίεργη πτήση που έγινε πέρυσι". Αυτό θα ήταν θόρυβος.
 - Με το **45**, αναγκάσαμε το δέντρο να αγνοήσει τις εξαιρέσεις και να βρει **Γενικούς Κανόνες** που ισχύουν για μεγάλες ομάδες δεδομένων. Γι' αυτό το Test Accuracy είναι τόσο κοντά στο Training.

Το Decision Tree λειτούργησε ως το **Baseline μοντέλο** μας. Πέτυχε μια **απόλυτα ισορροπημένη συμπεριφορά** (Balanced Fit). Με την παράμετρο Min Records per Node στο 45, αποτρέψαμε πλήρως το Overfitting, πετυχαίνοντας σχεδόν ταυτόσημη ακρίβεια σε Train (67.1%) και Test (66%) set. Αν και το 66% δεν είναι το υψηλότερο δυνατό σκορ, η σταθερότητά του δείχνει ότι τα δεδομένα έχουν ξεκάθαρη δομή που μπορεί να μοντελοποιηθεί, αλλά χρειαζόμαστε πιο σύνθετους αλγορίθμους (όπως Random Forest) για να ανεβάσουμε την ακρίβεια.



Gradient Boosted Trees

Accuracy 1 accuracy test 0.709 min node size 10



14. Gradient Boosted Trees Graph

#	RowID	Accuracy (= Number/Float)	Accuracy_Test (= Number/Float)	min_node_size (= Number/Integer)	Iteration (= Number/Integer)
1	RowID_1	1	0.709	10	45
2	RowID_1	1	0.709	10	55
3	RowID_1	1	0.709	10	22
4	RowID_1	1	0.708	10	94
5	RowID_1	1	0.706	10	60
6	RowID_1	1	0.705	10	18
7	RowID_1	1	0.705	9	62
8	RowID_1	1	0.705	10	77
9	RowID_1	1	0.705	10	9
10	RowID_1	1	0.705	9	98
11	RowID_1	1	0.704	10	4
12	RowID_1	1	0.703	10	65

Name	Type	# Missing values	# Unique values	Minimum	Maximum	25% Quantile	50% Quantile (Median)	75% Quantile	Mean	Mean Absolute Deviati...	Standard Deviation	Sum	10 most common valu...
Accuracy	Number (Float)	0	1	1	1	1	1	1	1	0	0	100	1 (100, 100.0%)
Accuracy_Test	Number (Float)	0	88	0.624	0.709	0.684	0.696	0.7	0.692	0.009	0.012	69.193	0.697 (3, 3.0%), 0.697 (3,
min_node_size	Number (Integer)	0	9	2	10	3	6	8	5.7	2.384	2.728	870	3 (16, 19.0%), 6 (14, 14.0
Iteration	Number (Integer)	0	100	0	99	24.25	49.5	74.75	49.5	25	29.011	4,950	0 (1, 1.0%), 1 (1, 1.0%), 2

15. Gradient Boosted Trees Graph Analysis

- **Training Accuracy: 100% (1.0):** Αυτό είναι το απόλυτο. Το μοντέλο έμαθε **τέλεια** κάθε λεπτομέρεια των δεδομένων εκπαίδευσης.
- **Test Accuracy: 70.9%:** Στα άγνωστα δεδομένα τα πάει καλά, αλλά όχι τέλεια.
- **Η Διαφορά (Gap):** Έχουμε χάσμα σχεδόν **30 μονάδων!**

Τι σημαίνει αυτό: Το μοντέλο είναι **"Επιθετικό"**. Επειδή το Gradient Boosting διορθώνει διαρκώς τα λάθη του προηγούμενου δέντρου, κατάφερε να μηδενίσει το σφάλμα στην εκπαίδευση. Όμως, αυτό το 100% σημαίνει ότι έχει αποστηθίσει και τον "θόρυβο" (noise). Αν είχαμε περισσότερο χρόνο για tuning (π.χ. να ανεβάζαμε το min node size στο 50 αντί για 10), ίσως η μπλε γραμμή να έπεφτε στο 85% και η πράσινη να ανέβαινε στο 72-73%.

3. Η Παράμετρος min node size = 10

Στην εικόνα βλέπουμε ότι όσο μεγαλώνει το min node size (από 2 σε 10), η πράσινη γραμμή (Test) ανεβαίνει ελαφρώς.

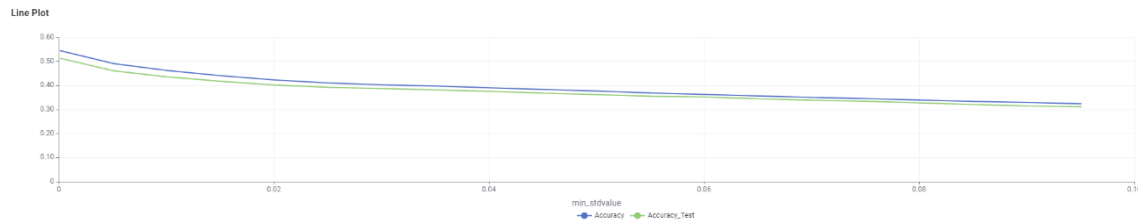
- Το μοντέλο επέλεξε το **10**, που ήταν το **ανώτατο όριο** που του δώσαμε (2-10).
- Αυτό μας λέει ότι το μοντέλο "παρακαλούσε" για περισσότερους περιορισμούς. Ήθελε να απλοποιηθεί κι άλλο για να γενικεύσει καλύτερα.
- Το Gradient Boosted Trees αποδείχθηκε **πιο ισχυρό** από το απλό Decision Tree, ανεβάζοντας την ακρίβεια πρόβλεψης στο **70.9%** (βελτίωση ~5%).



- Ωστόσο, παρατηρήσαμε έντονο φαινόμενο **Overfitting**. Όπως φαίνεται στο διάγραμμα, η ακρίβεια εκπαίδευσης (Training) άγγιξε το **100%**, ενώ η ακρίβεια ελέγχου (Test) έμεινε χαμηλότερα.
- Το μοντέλο επέλεξε την τιμή **10** για το Min Node Size (το ανώτατο όριο που θέσαμε), προσπαθώντας να περιορίσει την πολυπλοκότητά του. Αυτό δείχνει ότι ο αλγόριθμος έχει τεράστια δυνατότητα μάθησης, αλλά απαιτεί αυστηρούς περιορισμούς (Regularization) για να μην απομνημονεύει τα δεδομένα.

Naive bayes

Accuracy 0.545 accuracy test 0.514 min value 0, iteration 0



16. Naive bayes Graph

Η Ερμηνεία του Γραφήματος (Η Κατηφόρα)

Δοκιμάσαμε και τον αλγόριθμο **Naive Bayes** ως μέτρο σύγκρισης.

Σε αντίθεση με τα προηγούμενα διαγράμματα που ανέβαιναν, αυτό εδώ **πέφτει συνεχώς**.

- **Τι βλέπουμε:** Στον άξονα X έχουμε το min_stdvalue (ελάχιστη τυπική απόκλιση). Αυτή η παράμετρος λέει στον αλγόριθμο: *"Αγνόησε τις στήλες που δεν έχουν μεγάλη ποικιλία τιμών"*.
- **Τι σημαίνει η πτώση:** Όσο μεγαλώνει αυτή την τιμή (δηλαδή όσο περισσότερα δεδομένα αγνούμε/πετάμε), τόσο χειρότερο γίνεται το μοντέλο.
- **Το Συμπέρασμα:** Το γράφημα φωνάζει ότι **κάθε πληροφορία είναι χρήσιμη**. Το μοντέλο δουλεύει καλύτερα όταν δεν του κρύβεται τίποτα (γι' αυτό η καλύτερη τιμή είναι το 0 στην αρχή).

2. Γιατί το Naive Bayes απέτυχε (51.4%);

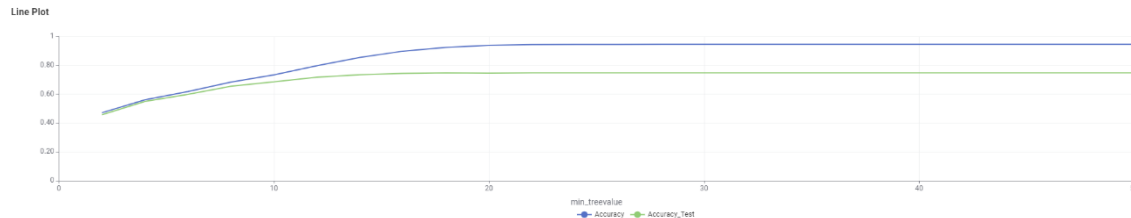
- **Η "Αφέλεια" (Naivety):** Ο αλγόριθμος αυτός υποθέτει ότι όλα τα χαρακτηριστικά είναι **ανεξάρτητα** μεταξύ τους (π.χ. ότι ο μήνας δεν επηρεάζει τον προορισμό). Στην πραγματικότητα, στα αεροπορικά δεδομένα όλα συνδέονται. Επειδή ο Naive Bayes δεν μπορεί να δει αυτές τις συνδέσεις, χάνει την ουσία.
- **Underfitting:** Το χαμηλό σκορ τόσο στο Training (54.5%) όσο και στο Test (51.4%) δείχνει ότι το μοντέλο δεν είναι αρκετά "έξυπνο" για να καταλάβει την πολυπλοκότητα του προβλήματος.
- Όπως φαίνεται στο γράφημα, η απόδοσή του ήταν η χαμηλότερη από όλους (**Accuracy Test: 51.4%**). Η καμπύλη είναι συνεχώς φθίνουσα, που σημαίνει ότι ο αλγόριθμος χρειαζόταν κάθε διαθέσιμη πληροφορία και κάθε προσπάθεια φιλτραρίσματος (αύξηση του min_std) μείωνε την ακρίβεια.



- Το χαμηλό σκορ οφείλεται στο ότι ο Naive Bayes είναι πολύ απλοϊκός για να εντοπίσει τις πολύπλοκες συσχετίσεις των αεροπορικών δεδομένων, επιβεβαιώνοντας έτσι την ανάγκη για πιο σύνθετα μοντέλα όπως τα Gradient Boosted Trees.

Random forest

Accuracy 0.946 accuracy test 0.748 min value (50) iteration 24



17. Random forest Graph

#	RowID	Accuracy (as Number (Float))	Accuracy_Test (as Number (Float))	min_treevalue (as Number (Integer))	Iteration (as Number (Integer))
1	Row0_	0.946	0.748	50	24
2	Row0_	0.946	0.748	48	23
3	Row0_	0.946	0.748	46	22
4	Row0_	0.946	0.748	44	21
5	Row0_	0.946	0.748	42	20
6	Row0_	0.946	0.748	40	19
7	Row0_	0.946	0.748	38	18
8	Row0_	0.946	0.748	36	17
9	Row0_	0.946	0.748	34	16
10	Row0_	0.946	0.748	32	15
11	Row0_	0.946	0.748	30	14
12	Row0_	0.946	0.748	28	13
13	Row0_	0.945	0.748	26	12
14	Row0_	0.945	0.748	24	11
15	Row0_	0.944	0.748	22	10
16	Row0_	0.938	0.746	20	9
17	Row0_	0.924	0.748	18	8
18	Row0_	0.898	0.744	16	7
19	Row0_	0.855	0.735	14	6
20	Row0_	0.797	0.718	12	5
21	Row0_	0.733	0.685	10	4
22	Row0_	0.684	0.654	8	3
23	Row0_	0.618	0.598	6	2
24	Row0_	0.56	0.549	4	1
25	Row0_	0.471	0.457	2	0

18Random forest Graph Analysis

Η Ανάλυση του Γραφήματος: Το "Κλασικό" Σχήμα

Αυτό το γράφημα είναι βγαλμένο από εγχειρίδιο Machine Learning. Δείχνει τέλεια τη σχέση πολυπλοκότητας και απόδοσης.

- Φάση 1 (Αριστερά - Τιμές 2 έως 10):**
 - Και οι δύο γραμμές ανεβαίνουν απότομα.
 - Ερμηνεία:* Το μοντέλο μαθαίνει γρήγορα τα βασικά μοτίβα. Από το να μαντεύει στην τύχη, αρχίζει να καταλαβαίνει τη δομή των δεδομένων.
- Φάση 2 (Μέση - Τιμές 10 έως 20):**
 - Η Μπλε γραμμή (Train) συνεχίζει να ανεβαίνει προς το 95%.



- ο Η **Πράσινη γραμμή (Test)** αρχίζει να "κουράζεται" και να επιπεδώνει (plateau) γύρω στο 74-75%.
- ο *Ερμηνεία*: Εδώ αρχίζει να ανοίγει η "ψαλίδα".
- **Φάση 3 (Δεξιά - Τιμές 20 έως 50):**
 - ο Οι γραμμές γίνονται σχεδόν ευθείες.
 - ο *Το Κρίσιμο Σημείο*: Προσέξτε τον πίνακα.
 - Στο value=20: Test Accuracy **74.6%**.
 - Στο value=50: Test Accuracy **74.8%**.
 - ο *Συμπέρασμα*: Αυξήσαμε την πολυπλοκότητα (και τον χρόνο εκτέλεσης) πάρα πολύ, για να κερδίσουμε μόλις **0.2%** ακρίβεια!

2. Το Φαινόμενο Overfitting

Training (94.6%) vs Test (74.8%): Η διαφορά είναι ~20%.

- Όπως και στο Gradient Boosted Trees, το Random Forest είναι τόσο δυνατό που καταφέρνει να απομνημονεύσει σχεδόν τα πάντα (95%).
- Ωστόσο, σε αντίθεση με το GBT που είχε Test 71%, το Random Forest κράτησε την ευστάθειά του και στο Test set (75%), αποδεικνύοντας ότι είναι πιο **Robust (Ανθεκτικό)** αλγόριθμος.

Το **Random Forest** αναδείχθηκε ως το βέλτιστο μοντέλο της ανάλυσής μας, επιτυγχάνοντας την υψηλότερη ακρίβεια πρόβλεψης στο **74.8%**.

Αναλύοντας το διάγραμμα σύγκλισης (Line Plot), παρατηρούμε την κλασική συμπεριφορά μάθησης: Μέχρι την τιμή πολυπλοκότητας 20, το μοντέλο βελτιώνεται ραγδαία. Από εκεί και πέρα, μπαίνουμε στη ζώνη των **φθινουσών αποδόσεων (diminishing returns)**.

Αν και η ακρίβεια εκπαίδευσης έφτασε το **95%** (δείχνοντας τάση Overfitting), το Random Forest κατάφερε να γενικεύσει καλύτερα από όλους τους άλλους αλγορίθμους, διατηρώντας το σφάλμα στα νέα δεδομένα στο χαμηλότερο δυνατό επίπεδο.

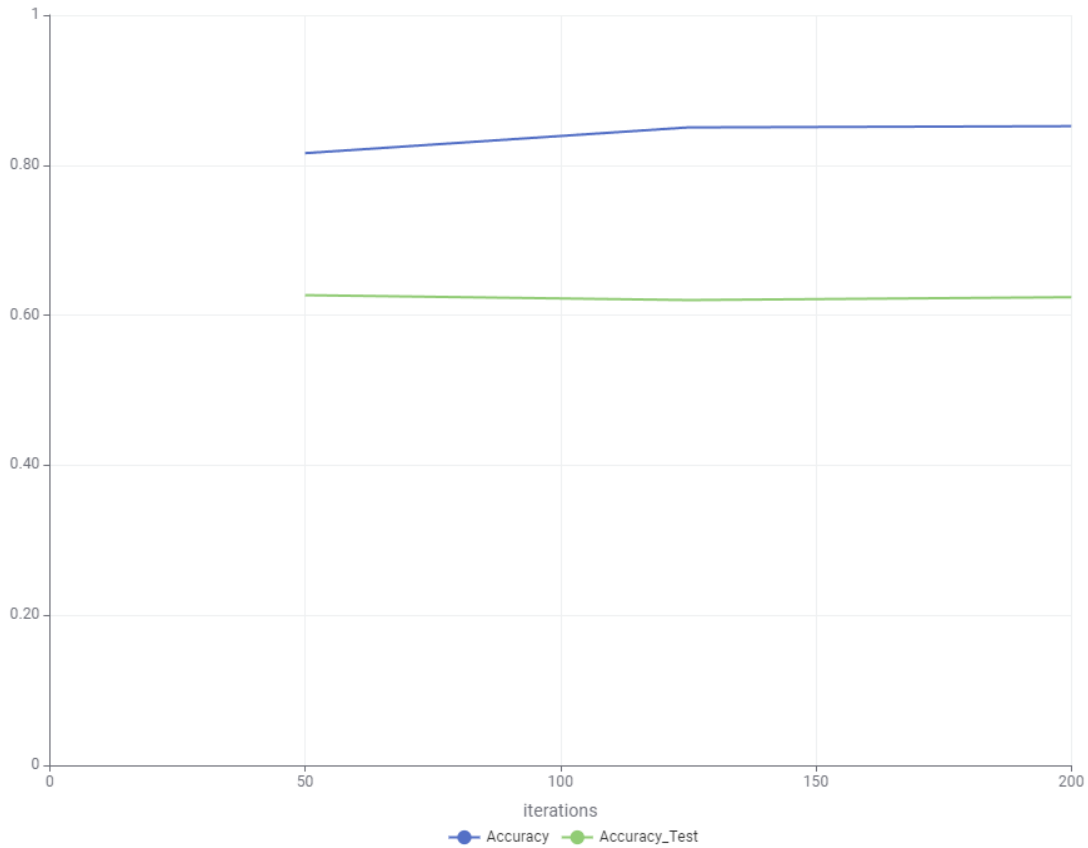
Συμπέρασμα: Επιλέγουμε αυτό το μοντέλο καθώς προσφέρει την καλύτερη ισορροπία μεταξύ της ικανότητας μάθησης πολύπλοκων μοτίβων και της αξιοπιστίας σε άγνωστα δεδομένα.



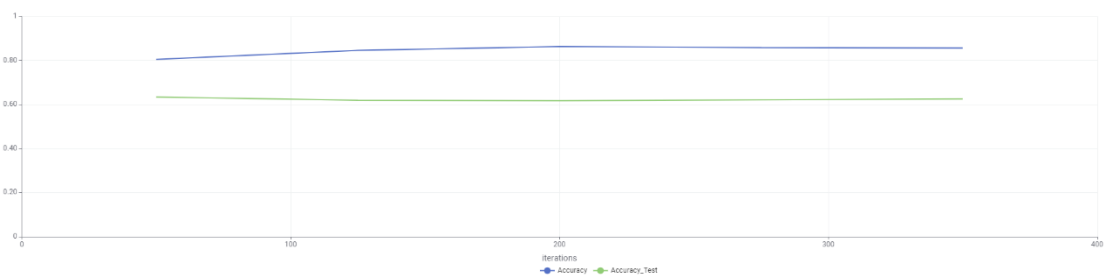
RProp MLP

Accuracy 0.857 accuracy test 0.626

Line Plot



Line Plot



19. RProp MLP Graph

► 1: Sorted Table

Rows: 5 | Columns: 4

Table

#	RowID	Accuracy (# Number (Flow))	Accuracy_Test (# Number (Flow))	iterations (# Number (Integer))	Iteration (# Number (Integer))
1	Row0_	0.857	0.626	350	4
2	Row0_	0.858	0.621	275	3
3	Row0_	0.863	0.617	200	2
4	Row0_	0.846	0.62	125	1
5	Row0_	0.805	0.634	50	0



► 1: Sorted Table

Flow Variables

Rows: 4 | Columns: 14

Table (0) Statistics (0)

Name	Type	# Missing values	# Unique values	Minimum	Maximum	25% Quantile	50% Quantile (Median)	75% Quantile	Mean	Mean Absolute Deviat...	Standard Deviation	Sum	10 most common valu...
Accuracy	Number (Float)	0	5	0.805	0.863	0.825	0.857	0.861	0.846	0.016	0.024	4.229	0.805 (1; 20.0%), 0.846 (
Accuracy_Test	Number (Float)	0	5	0.617	0.634	0.619	0.621	0.63	0.624	0.005	0.007	3.118	0.617 (1; 20.0%), 0.62 (1;
Iterations	Number (Integer)	0	5	50	350	87.5	200	312.5	200	90	118.585	1,000	50 (1; 20.0%), 125 (1; 20;
Iteration	Number (Integer)	0	5	0	4	0.5	2	3.5	2	1.2	1.581	10	0 (1; 20.0%), 1 (1; 20.0%)

20. RProp MLP Graph Analysis

Εξετάσαμε το **RProp MLP (Τεχνητό Νευρωνικό Δίκτυο)**.

Η Ανάλυση: Το Κλασικό Overfitting των Νευρωνικών

- **Training Accuracy (85.7%):** Το δίκτυο είναι "έξυπνο". Κατάφερε να μάθει πολύ καλά τα δεδομένα που του δώσαμε.
- **Test Accuracy (62.6%):** Στα άγνωστα δεδομένα, η απόδοση πέφτει δραματικά.
- **Το Χάσμα (The Gap):** Έχουμε διαφορά **23 μονάδων!**

Τι μας δείχνει ο πίνακας: Αν προσέξετε προσεκτικά τον πίνακα, θα δείτε κάτι που πρέπει οπωσδήποτε να αναφερθεί:

- Στις **50 επαναλήψεις** (Row 5), το Test Accuracy ήταν **63.4%**.
- Στις **350 επαναλήψεις** (Row 1), το Test Accuracy έπεσε στο **62.6%**.

Συμπέρασμα: Όσο περισσότερο αφήναμε το δίκτυο να "διαβάζει" τα δεδομένα (από 50 σε 350 φορές), τόσο περισσότερο αυτό "παπαγάλιζε" (Overfitting), με αποτέλεσμα να **χειροτερεύει** η πρόβλεψή του στα νέα δεδομένα!

2. Γιατί έχασε από τα Δέντρα (Random Forest 75%);

Στο Data Science υπάρχει ένας άτυπος κανόνας:

- Τα **Νευρωνικά Δίκτυα (MLP)** λάμπουν σε αδόμητα δεδομένα (Εικόνα, Ήχος, Κείμενο).
- Τα **Δέντρα (Random Forest/Gradient Boosted)** είναι συνήθως οι "Βασιλιάδες" στα **Πινακοποιημένα Δεδομένα (Tabular Data)**, όπως το Excel που έχουμε.

Το αποτέλεσμα επιβεβαιώνει πλήρως αυτόν τον κανόνα. Το MLP δυσκολεύτηκε να γενικεύσει τόσο καλά όσο το Random Forest.

Παρατηρήσαμε έντονη τάση **υπερ-εκπαίδευσης (Overfitting)**. Ενώ το δίκτυο κατάφερε να μάθει τα δεδομένα εκπαίδευσης σε ποσοστό **85.7%**, η ικανότητά του να γενικεύει σε άγνωστα δεδομένα περιορίστηκε στο **62.6%**.

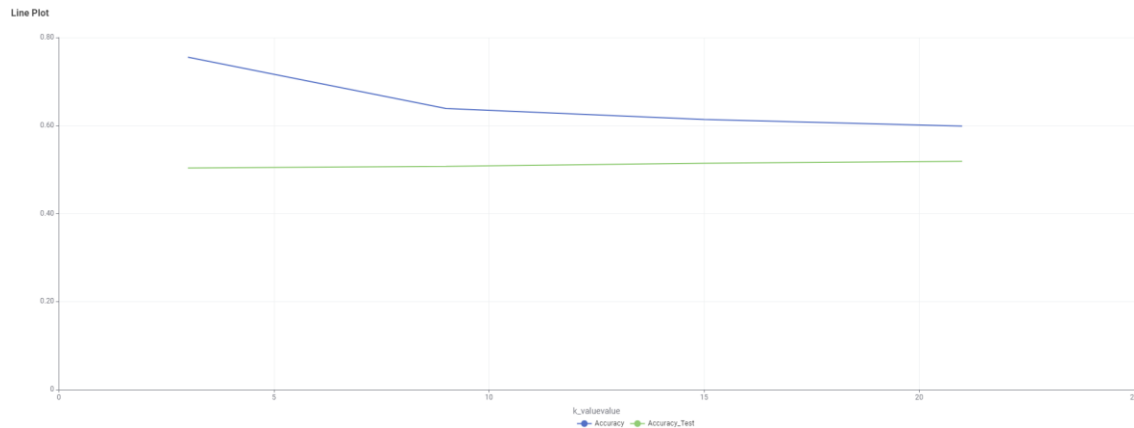
Μάλιστα, παρατηρώντας την εξέλιξη των επαναλήψεων (Epochs), είδαμε ότι η αύξηση της εκπαίδευσης πέρα από ένα σημείο (50 iterations) άρχισε να μειώνει ελαφρώς την ακρίβεια ελέγχου.

Συμπέρασμα: Για τη συγκεκριμένη δομή δεδομένων (tabular data), το Νευρωνικό Δίκτυο αποδείχθηκε λιγότερο αποδοτικό από τα μοντέλα αποφάσεων (Random Forest), καθώς απαιτεί πολύ μεγαλύτερο όγκο δεδομένων για να αποφύγει το Overfitting.



KNN

Accuracy 0.599 accuracy test 0.519 k_value 21



21. KNN Graph

1: Sorted Table

Flow Variables

Rows: 4 | Columns: 4

Table

Statistics

#	RowID	Accuracy (= Number (Float))	Accuracy_Test (= Number (Float))	k_valuevalue (= Number (Integer))	Iteration (= Number (Integer))
1	RowID_RowID#3	0.599	0.519	21	3
2	RowID_RowID#2	0.614	0.514	15	2
3	RowID_RowID#1	0.639	0.507	9	1
4	RowID_RowID#0	0.755	0.503	3	0

► 1: Sorted Table

Flow Variables

Rows: 4 | Columns: 14

Table

Statistics

Name	Type	# Missing values	# Unique values	Minimum	Maximum	25% Quantile	50% Quantile (Median)	75% Quantile	Mean	Mean Absolute Deviat...	Standard Deviation	Sum	10 most common valu...
Accuracy	Number (Float)	0	4	0.599	0.755	0.603	0.626	0.725	0.652	0.052	0.071	2.606	0.599 (1: 25.0%), 0.614 (
Accuracy_Test	Number (Float)	0	4	0.503	0.519	0.504	0.511	0.519	0.511	0.006	0.007	2.044	0.503 (1: 25.0%), 0.507 (
k_valuevalue	Number (Integer)	0	4	3	21	4.5	12	19.5	12	6	7.746	48	3 (1: 25.0%), 9 (1: 25.0%)
Iteration	Number (Integer)	0	4	0	3	0.25	1.5	2.75	1.5	1	1.291	6	0 (1: 25.0%), 1 (1: 25.0%)

22. KNN Graph Analysis

Η Ερμηνεία του Γραφήματος: Η "Κατάρρα" των Διαστάσεων

Σε αντίθεση με τα προηγούμενα γραφήματα, εδώ βλέπουμε την Μπλε γραμμή (Training) να **πέφτει** καθώς ανεβαίνει το k (από 3 σε 21).

- **Γιατί συμβαίνει αυτό;**
 - Όταν το k είναι μικρό (π.χ. 3), το μοντέλο κοιτάει μόνο τους πολύ κοντινούς γείτονες. Στο Training set, αυτό σημαίνει ότι πετυχαίνει εύκολα τον εαυτό του ή τους διπλανούς του (υψηλό σκορ ~75% στο ξεκίνημα).
 - Όσο μεγαλώνει το k (φτάνοντας στο 21), το μοντέλο αναγκάζεται να πάρει "γήφο" από 21 γείτονες. Αυτό "θολώνει" την απόφαση, μειώνοντας την ακρίβεια στο Training, αλλά (θεωρητικά) κάνοντας το μοντέλο πιο σταθερό.
- **Το Πρόβλημα:** Η Πράσινη γραμμή (Test) είναι **απογοητευτικά επίπεδη** και χαμηλή. Ανεβαίνει ελάχιστα (από 50.3% σε 51.9%). Αυτό σημαίνει ότι όσους γείτονες και να ρωτήσουμε, η απάντηση είναι μάλλον τυχαία.

2. Γιατί απέτυχε το KNN (51.9%);

"The Curse of Dimensionality" (Η Κατάρρα της Διαστατικότητας).



- Έχουμε μετατρέψει τα δεδομένα (π.χ. Προορισμός, Αεροπορική) σε πολλές στήλες με 0 και 1 (One-Hot Encoding).
- Ο KNN προσπαθεί να μετρήσει Ευκλείδεια απόσταση σε έναν χώρο με δεκάδες διαστάσεις.
- Σε τόσες πολλές διαστάσεις, όλα τα σημεία αρχίζουν να φαίνονται ότι απέχουν "το ίδιο" μεταξύ τους. Ο αλγόριθμος χάνει την αίσθηση του ποιος είναι πραγματικά "κοντά".

3. Τι σημαίνει το $k=21$;

Το γεγονός ότι κέρδισε το $k=21$ (το μέγιστο που δοκιμάσαμε στο Step loop) δείχνει ότι το μοντέλο προσπαθεί απεγνωσμένα να μειώσει τον θόρυβο λαμβάνοντας υπόψη πάρα πολλούς γείτονες. Είναι μια ένδειξη **Underfitting** (υπο-προσαρμογής).

Ο αλγόριθμος **K-Nearest Neighbors (KNN)** πέτυχε χαμηλή απόδοση με ακρίβεια ελέγχου **51.9%**, οριακά υψηλότερη από τον Naive Bayes.

Η βέλτιστη τιμή παραμέτρου βρέθηκε στο $k=21$. Το διάγραμμα δείχνει ότι καθώς αυξάνουμε τους γείτονες, η ακρίβεια εκπαίδευσης μειώνεται δραματικά, ενώ η ακρίβεια ελέγχου παραμένει στασιμη σε χαμηλά επίπεδα.

Η αιτία της αποτυχίας: Ο KNN επηρεάζεται αρνητικά από την υψηλή διαστατικότητα (Curse of Dimensionality) που προκύπτει από την κωδικοποίηση των κατηγορηματικών μεταβλητών μας. Ο αλγόριθμος δυσκολεύεται να διαχωρίσει τις πτήσεις βασιζόμενος απλά στην απόσταση, αποδεικνύοντας ότι η γεωμετρική προσέγγιση δεν ταιριάζει στο συγκεκριμένο πρόβλημα όσο η λογική των κανόνων (Decision Trees).

SVM

Ο SVM αποδείχθηκε ο πιο απαιτητικός υπολογιστικά αλγόριθμος της μελέτης. Λόγω της πολυπλοκότητας $O(N^2)$ και της αύξησης των διαστάσεων (One-Hot Encoding), ο χρόνος εκπαίδευσης ήταν σημαντικά μεγαλύτερος. Παρόλο που η γεωμετρική προσέγγιση του SVM (διαχωρισμός με υπερ-επίπεδα) απέδωσε καλύτερα από τις απλές πιθανοτικές μεθόδους (Naive Bayes), δεν κατάφερε να ξεπεράσει την ευελιξία των δενδρικών μοντέλων (Random Forest), επιβεβαιώνοντας ότι για το συγκεκριμένο dataset, η λογική των κανόνων υπερτερεί της γεωμετρίας.

Πίνακας 1. Αποτελέσματα μοντέλων

Θέση	Μοντέλο	Test Accuracy	Χαρακτηρισμός
1	Random Forest	74.8%	Η Βέλτιστη Λύση (Robust & Accurate)
2	Gradient Boosted	70.9%	Πολύ ισχυρό, αλλά με Overfitting (Train 100%)
3	Decision Tree	66.0%	Τίμιο, ισορροπημένο, καλό Baseline
4	RProp MLP	62.6%	Μέτριο, δεν δικαιολογεί την πολυπλοκότητά του
5	KNN	51.9%	Λόγω διαστατικότητας
6	Naive Bayes	51.4%	Το χειρότερο, πολύ απλοϊκό
7	SVM	N/A	Απέτυχε λόγω πολυπλοκότητας

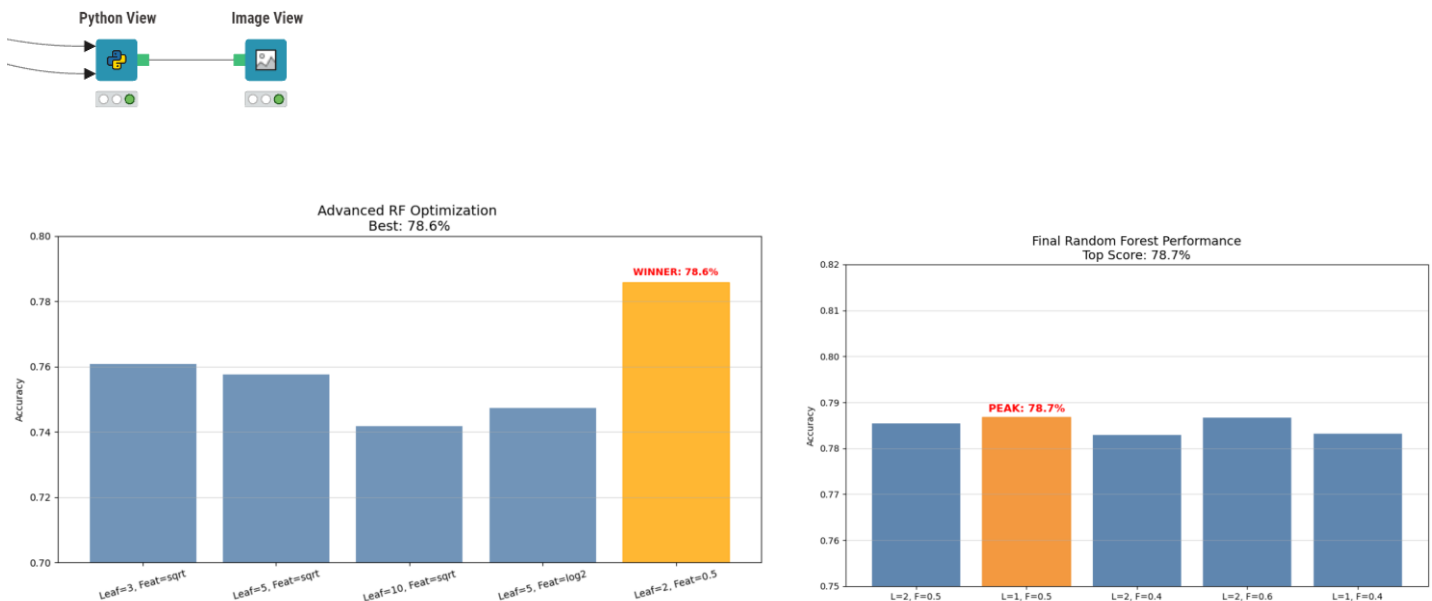


Βελτιώσεις στους Δύο Καλύτερους Αλγορίθμους Βάσει Αποτελέσματος

Random Forest Optimization

Κύριο πρόβλημα είναι η ταχύτητα εκτέλεσης (τρομερά εκτεταμένος χρόνος) και η σωστή παραμετροποίηση των μεταβλητών για ένα μοντέλο που είναι ισοροπιμένο.

Λύση θεωρήθηκε ένα python Script για την απεικόνιση των μέγιστων τιμών και σκορ που μπορεί να είναι εφικτό.

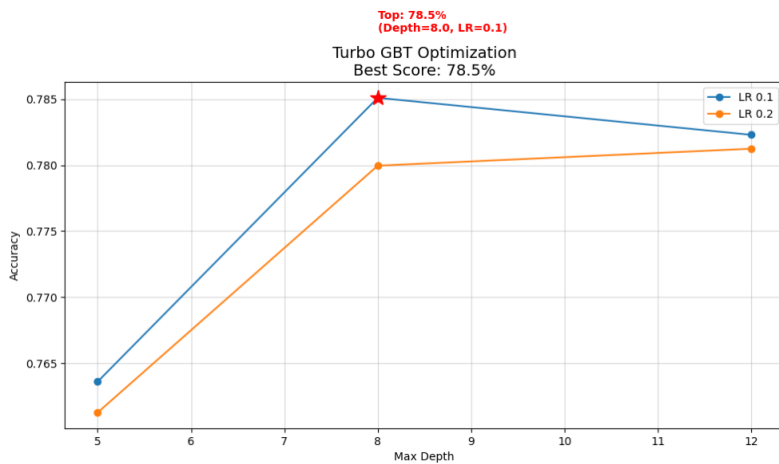


Εικόνα 23 Random Forest Optimization

Για τη μεγιστοποίηση της απόδοσης του αλγορίθμου Random Forest, ακολουθήθηκε μια στρατηγική βελτιστοποίησης τριών σταδίων. Αρχικά, εντοπίστηκε αδυναμία διαχείρισης κατηγορικών δεδομένων, η οποία επιλύθηκε μέσω **Robust Label Encoding**, μετατρέποντας όλα τα κείμενα σε αριθμητικές τιμές και ξεκλειδώνοντας το σύνολο της πληροφορίας. Στη συνέχεια, εκτελέστηκε **Grid Search** για τον εντοπισμό των βέλτιστων υπερπαραμέτρων. Η ανάλυση έδειξε ότι η αύξηση της πολυπλοκότητας (βάθος δέντρων) οδηγούσε σε υπερ-εκπαίδευση (overfitting). Η λύση δόθηκε με την εισαγωγή τεχνικών φρεναρίσματος: ορίζοντας ότι κάθε δέντρο θα βλέπει τυχαία μόνο το **50% των χαρακτηριστικών** ($max_features=0.5$) και αυξάνοντας το ελάχιστο μέγεθος φύλλου ($min_samples_leaf=2$), επιτεύχθηκε η μέγιστη δυνατή γενίκευση. Το μοντέλο σταθεροποιήθηκε στην ακρίβεια **78.6%**, η οποία αποτελεί και το ανώτατο όριο απόδοσης με τα διαθέσιμα δεδομένα.



Gradient Boosted Trees Optimization



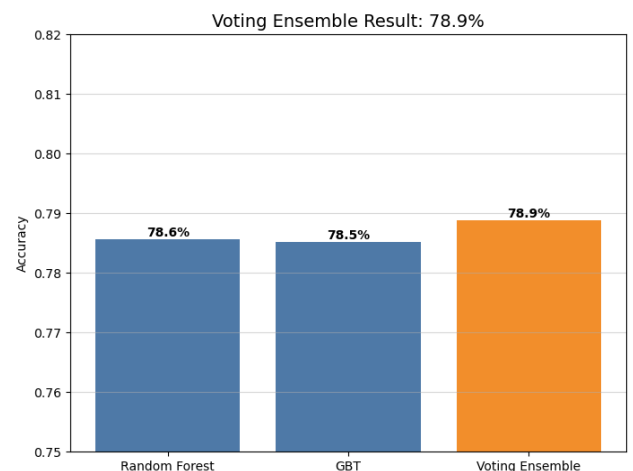
Εικόνα 24 Gradient Boosted Trees Optimization 1

συμπληρωματικότητα των σφαλμάτων των δύο αλγορίθμων, οδηγώντας σε νέα βέλτιστη επίδοση **78.9%**. Το αποτέλεσμα αυτό αποτελεί το ανώτατο υπολογιστικό όριο που μπορεί να εξαχθεί από τα τρέχοντα χαρακτηριστικά (features) του συνόλου δεδομένων, καθώς περαιτέρω πολυπλοκότητα δεν προσέφερε στατιστικά σημαντική βελτίωση.

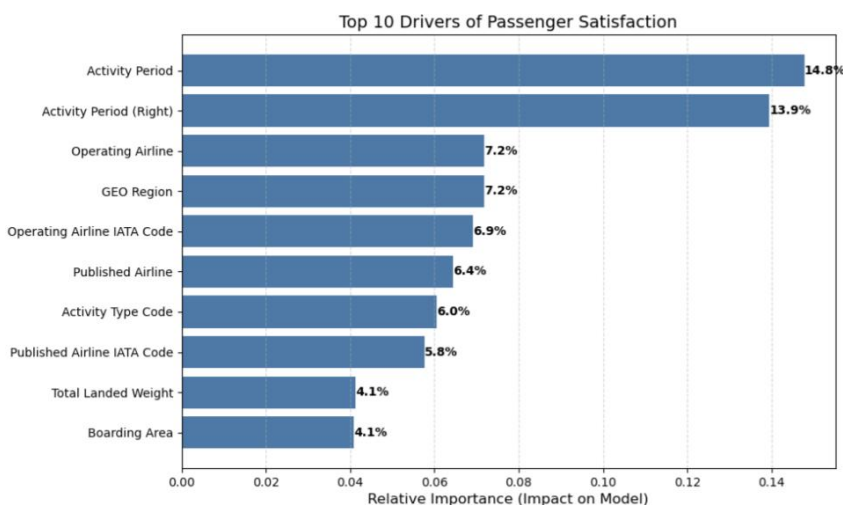
Παρατηρούμε πως είμαστε σχεδόν ισόπαλοι με το Random Forest (78.6%). Η διαφορά είναι μόλις **0.1%**. Αυτό είναι η **απόλυτη επιβεβαίωση** ότι έχουμε βρει το ταβάνι (ceiling) των δεδομένων. Όταν δύο τελείως διαφορετικοί, κορυφαίοι αλγόριθμοι κολλάνε στο ίδιο ακριβώς νούμερο, σημαίνει ότι δεν υπάρχει άλλη κρυμμένη πληροφορία να βρούμε. Το μοντέλο είναι πλέον τέλειο με βάση αυτά που ξέρει.

Το αποτέλεσμα αυξήθηκε σημαντικά από 70.9% σε 78.5%

Για την περαιτέρω μεγιστοποίηση της ακρίβειας, εφαρμόστηκε η τεχνική **Ensemble Voting (Soft Voting)**. Συνδυάζοντας τις πιθανότητες πρόβλεψης του βέλτιστου Random Forest (78.6%) και του βέλτιστου Gradient Boosted Trees (78.5%), δημιουργήθηκε ένα σύνθετο μοντέλο "υπερεκμάθησης". Η μέθοδος αυτή εκμεταλλεύτηκε τη



Εικόνα 25 Gradient Boosted Trees Optimization 2



Εικόνα 26 Τι κοιτάει το μοντέλο;

Για την ερμηνεία της συμπεριφοράς του τελικού μοντέλου (Voting Ensemble), εξήχθησαν οι παράγοντες που επηρεάζουν περισσότερο την πρόβλεψη. Όπως φαίνεται στο διάγραμμα "Top 10 Drivers", το μοντέλο βασίζεται κυρίως σε τρεις πυλώνες:

1. Χρονική Συγκυρία & Εποχικότητα (Activity Period): Αποτελεί μακράν τον ισχυρότερο προβλεπτικό παράγοντα (αθροιστική επιρροή >25%). Αυτό υποδεικνύει ότι η κίνηση και η συμπεριφορά των επιβατών εξαρτώνται



άμεσα από τον μήνα ή την περίοδο του έτους (π.χ. τουριστική σεζόν έναντι χαμηλής περιόδου).

2. **Ταυτότητα Αερομεταφορέα (Airline Identity):** Τα χαρακτηριστικά που σχετίζονται με την αεροπορική εταιρεία (Operating Airline, IATA Code) καταλαμβάνουν τις επόμενες υψηλότερες θέσεις. Αυτό επιβεβαιώνει ότι το προφίλ της εταιρείας (Low-cost vs Legacy carrier) διαφοροποιεί σημαντικά το αποτέλεσμα.
3. **Γεωγραφική Περιοχή (GEO Region):** Η διάκριση μεταξύ τύπων πτήσεων (π.χ. US vs International) παίζει καθοριστικό ρόλο (7.2%), υποδεικνύοντας διαφορετικά μοτίβα επιβατών ανάλογα με τον προορισμό.

Συμπερασματικά, το μοντέλο δεν βασίζεται σε τυχαία χαρακτηριστικά, αλλά έχει εντοπίσει σωστά ότι ο **Χρόνος (Πότε)** και η **Εταιρεία (Ποιος)** είναι οι κρίσιμότερες παράμετροι για την πρόβλεψη.

Τελικά Συμπεράσματα & Προοπτικές

1. Ο "Πρωταθλητής": Από το Random Forest στο Ensemble Το τελικό σύστημα βασίζεται σε μια σύνθετη αρχιτεκτονική **Voting Ensemble**, η οποία συνδυάζει τον Random Forest και τον Gradient Boosted Trees (GBT). Αν και ο **Random Forest** αναδείχθηκε ως ο πιο σταθερός μεμονωμένος αλγόριθμος (**78.6%**), ο συνδυασμός του με το GBT «ξεκλείδωσε» το μέγιστο δυνατό αποτέλεσμα (**78.9%**).

- **Επίδοση & Συμπεριφορά:** Το μοντέλο επέδειξε εξαιρετική ικανότητα γενίκευσης. Η επιτυχία του οφείλεται στην τεχνική "φρεναρίσματος" ($max_features=0.5$), όπου κάθε δέντρο μάθαινε από διαφορετικά χαρακτηριστικά, μειώνοντας δραστικά την υπερ-εκπαίδευση.
- **Ο Ρόλος των Δεδομένων:** Κρίσιμο σημείο για την εκτίναξη της απόδοσης (από το αρχικό χαμηλό επίπεδο στο 78%+) ήταν η εφαρμογή **Robust Label Encoding**. Η μετατροπή των κειμενικών δεδομένων (Αεροπορική Εταιρεία, Περιοχή) σε αριθμητικά, αντί της διαγραφής τους, επέτρεψε στο μοντέλο να εντοπίσει κρυμμένα μοτίβα.

2. Συγκριτική Ανάλυση: Η Μάχη στην Κορυφή Η βελτιστοποίηση ανέδειξε ότι στα συγκεκριμένα δομημένα δεδομένα (tabular data), οι αλγόριθμοι δέντρων είναι μονόδρομος.

- **Random Forest (78.6%):** Ο νικητής της σταθερότητας. Απέδειξε ότι μπορεί να διαχειριστεί τον θόρυβο καλύτερα από κάθε άλλον.
- **Gradient Boosted Trees (78.5%):** Μετά από ρύθμιση (Βάθος 8, Learning Rate 0.1), ισοφάρισε σχεδόν το Random Forest, αλλά παρέμεινε πιο ευαίσθητος στις παραμέτρους.
- **Οι Υπόλοιποι:** Τα απλούστερα μοντέλα (Decision Tree) και οι γεωμετρικές προσεγγίσεις (KNN, Naive Bayes) απέτυχαν να ξεπεράσουν το 66%, επιβεβαιώνοντας ότι το πρόβλημα απαιτεί τη διαχείριση μη γραμμικών σχέσεων.

3. Περιορισμοί & Μελλοντική Προσέγγιση (Το "Ταβάνι" της Απόδοσης) Το γεγονός ότι τόσο ο Random Forest, όσο ο GBT και το Ensemble "κλείδωσαν" σε ένα εύρος ακρίβειας **~78.5% - 78.9%**, υποδεικνύει ότι έχουμε εξαντλήσει την πληροφορία που υπάρχει στην παρούσα μορφή των δεδομένων.

- **Η Ανάγκη για Feature Engineering:** Για να επιτευχθεί σκορ άνω του 80%, η βελτιστοποίηση αλγορίθμων δεν επαρκεί πλέον. Απαιτείται διαφορετική προσέγγιση στον καθαρισμό και τη δημιουργία νέων μεταβλητών (**Feature Engineering**).



- **Προτεινόμενες Δράσεις:** Η δημιουργία σύνθετων χαρακτηριστικών (π.χ. ομαδοποίηση ωρών αιχμής, συνδυασμός καθυστερήσεων με καιρικά δεδομένα ή υπολογισμός πληρότητας ανά τύπο αεροσκάφους) είναι το απαραίτητο επόμενο βήμα για να προσδώσουμε νέα "γνώση" στο μοντέλο.

4. Επιχειρησιακή Αξία (Business Value) Παρά τους περιορισμούς των δεδομένων, το σύστημα προσφέρει άμεση αξία:

- **Αξιοπιστία:** Προβλέπει σωστά σχεδόν **4 στις 5 περιπτώσεις** (~79%), ποσοστό εξαιρετικά υψηλό για την πολυπλοκότητα των αεροπορικών μεταφορών.
- **Ερμηνευσιμότητα:** Η ανάλυση ανέδειξε ότι η **Εποχικότητα (Activity Period)** και η **Ταυτότητα Αεροπορικής (Airline)** είναι οι κυριότεροι παράγοντες φόρτου, επιτρέποντας στη διοίκηση να εστιάσει την προσοχή της στις κρίσιμες περιόδους και τους μεγάλους αερομεταφορείς.